

# Module 08 — Search Trees & Augmentation

---

**Sources:** CLRS 4e ch. 12 (Binary Search Trees), ch. 13 (Red-Black Trees), ch. 17 (Augmenting Data Structures), ch. 18 (B-Trees) · Skiena 3e §3.4 (Binary Search Trees)

---

## Big Idea

A sorted array gives you  $O(\lg n)$  search but  $O(n)$  update. A linked list gives you  $O(1)$  update but  $O(n)$  search. **Search trees are the structure that refuses to choose.** Skiena frames the whole chapter as exactly this trade: binary search needs fast access to "the median elements above and below" a node, so build a linked list with *two* pointers per node and you get binary search over a structure you can still splice.

The single organizing property is the **binary-search-tree property**: for every node  $x$ , every key in  $x.\text{left}$  is  $\leq x.\text{key}$  and every key in  $x.\text{right}$  is  $\geq x.\text{key}$ . That one local invariant buys you *all* the dynamic-set operations — SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR, INSERT, DELETE — in  $O(h)$  time, plus sorted output for free via an inorder walk. Everything else in this module is about **controlling  $h$** .

And  $h$  is not automatically small. A plain BST built by insertion has a shape dictated entirely by the *insertion order*, which you do not control: sorted input gives you a linked list with extra pointers, height  $n$ . Two escapes exist:

1. **Rebalance as you go** — red-black trees color nodes and rotate locally so that no root-to-leaf path is more than twice as long as any other, forcing  $h \leq 2 \lg(n+1)$ .
2. **Widen the nodes** — B-trees put hundreds of keys in one node so the log's *base* becomes huge, which matters when a node access is a disk read.

The last idea is the one that pays off most in interviews and in production: a balanced BST is not just a dictionary, it is a **skeleton you can hang extra information on**. Store a subtree size in each node and you have order statistics ( $k$ -th smallest, rank of a key) in  $O(\lg n)$ . Store a subtree maximum endpoint and you have interval stabbing queries. CLRS ch. 17 turns this into a mechanical four-step recipe with a theorem attached (Theorem 17.1) that tells you exactly when the augmentation is free.

**Remember months later:** a BST is  $O(h)$  for everything; balancing is what turns  $h$  into  $\lg n$ ; augmentation is what turns a dictionary into a specialized query engine, and it is free whenever the new field at  $x$  is computable in  $O(1)$  from  $x$ ,  $x.\text{left}$ ,  $x.\text{right}$ .

---

## What You Should Be Able To Do After This Chapter

- State the BST property precisely and explain why an inorder walk prints sorted keys — and prove the  $\Theta(n)$  bound for the walk.
  - Implement search / minimum / maximum / successor / predecessor / insert / delete on a plain BST, including the three-case deletion, without looking anything up.
  - Explain why a BST's height ranges from  $\lceil \lg(n+1) \rceil$  to  $n$ , what determines which you get, and why random insertion order gives  $\Theta(\lg n)$  **with high probability**.
  - State the five red-black properties, define black-height, and reproduce the proof of Lemma 13.1 ( $h \leq 2 \lg(n+1)$ ).
  - Write `LEFT-ROTATE` from memory and explain why a rotation preserves the BST property in  $O(1)$  time.
  - Explain the three cases of `RB-INSERT-FIXUP` and the three-part loop invariant, and say *why* insertion does at most two rotations while deletion does at most three.
  - Explain the "extra black" / doubly-black device that makes `RB-DELETE-FIXUP` work, and identify which of the four cases is the only one that loops.
  - Apply CLRS's four-step augmentation method to a new problem, and state Theorem 17.1 and the condition it requires.
  - Implement an order-statistic tree (`OS-SELECT`, `OS-RANK`) and an interval tree (`INTERVAL-SEARCH`), and prove interval search is correct using the interval trichotomy.
  - Define a B-tree by minimum degree  $t$ , derive  $h \leq \log_t((n+1)/2)$ , and describe insertion by proactive splitting and deletion by proactive filling — and say why both are single downward passes.
  - Recognize, in an interview, when the answer is "balanced BST" rather than "hash table", "heap", or "sorted array".
- 

## Part 1 — Binary Search Trees (CLRS 12, Skiena 3.4)

## The problem search trees solve

### Unified Understanding

We want a **dictionary** — INSERT, DELETE, SEARCH — and often the **ordered** operations too: MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR, range queries, sorted output. Skiena's framing is the sharpest: we have seen structures that give *fast search* or *flexible update*, never both.

| STRUCTURE                   | SEARCH                  | INSERT                  | DELETE                  | SUCCESSOR   | SORTED OUTPUT     |
|-----------------------------|-------------------------|-------------------------|-------------------------|-------------|-------------------|
| Unsorted doubly-linked list | $\theta(n)$             | $O(1)$                  | $O(1)$ (given node)     | $\theta(n)$ | $\theta(n \lg n)$ |
| Sorted array                | $O(\lg n)$              | $\theta(n)$             | $\theta(n)$             | $O(1)$      | $\theta(n)$       |
| Hash table                  | $\theta(1)$<br>expected | $\theta(1)$<br>expected | $\theta(1)$<br>expected | $\theta(n)$ | $\theta(n \lg n)$ |
| <b>Balanced BST</b>         | $O(\lg n)$              | $O(\lg n)$              | $O(\lg n)$              | $O(\lg n)$  | $\theta(n)$       |

Skiena's insight into *why* the tree works: **binary search needs fast access to the median elements above and below the current node**. A sorted array gives you that by arithmetic ( $\text{mid} = (\text{lo} + \text{hi})/2$ ) but pays  $\theta(n)$  to keep the array sorted. If instead you *hard-wire* the "go left" and "go right" targets as pointers, you keep the binary search and regain  $O(1)$  splicing. That is literally all a BST is: a linked list with two pointers per node, wired so that following them performs binary search.

***Skiena emphasis:** the tree as "linked list with two pointers", motivated by the search/update tension; and the take-home lesson that picking the wrong structure is disastrous while picking the best among several good ones rarely is.*

***CLRS emphasis:** the tree as an implementation of the abstract **dynamic set** with a stated invariant, and everything proved from that invariant.*

### Definition and the BST property

A **rooted binary tree** is recursively either (1) empty, or (2) a root node plus a left and a right rooted binary tree. Order matters — left  $\neq$  right. (Skiena: there are exactly 5 distinct binary tree shapes on 3 nodes, and for *any* shape and *any* set of  $n$  keys there is **exactly one** labeling that makes it a binary search tree.)

**BST property.** Let  $x$  be a node.

- If  $y$  is in the left subtree of  $x$ , then  $y.key \leq x.key$ .
- If  $y$  is in the right subtree of  $x$ , then  $y.key \geq x.key$ .

Each node stores `key`, `left`, `right`, and (usually) `p` (parent). `NIL` marks absent children; `x.p == NIL` identifies the root.

***Duplicate keys.** Skiena, in a footnote worth memorizing: "Allowing duplicate keys in a binary search tree (or any other dictionary structure) is bad karma, often leading to very subtle errors." His fix is a third pointer per node holding a list of all items with that key. CLRS Problem 12-1 explores what happens if you don't: naively sending equal keys right gives  $\Theta(n^2)$  on  $n$  identical keys; a boolean "alternate direction" flag or a random direction restores  $\Theta(n \lg n)$  expected.*

## Inorder traversal

```
INORDER-TREE-WALK(x)
1  if x ≠ NIL
2      INORDER-TREE-WALK(x.left)
3      print x.key
4      INORDER-TREE-WALK(x.right)
```

→ **C++ implementation:** [A1 INORDER-TREE-WALK](#)

**Theorem 12.1.** If  $x$  is the root of an  $n$ -node subtree, `INORDER-TREE-WALK(x)` takes  $\Theta(n)$  time.

*Proof skeleton.* Let  $T(n)$  be the time.  $T(0) = c$  for the `NIL` test. For  $n > 0$  with  $k$  nodes in the left subtree and  $n-k-1$  in the right,  $T(n) = T(k) + T(n-k-1) + d$ . Guess  $T(n) = (c+d)n + c$  and verify by substitution: it holds for  $n = 0$ , and

$$T(n) = ((c+d)k + c) + ((c+d)(n-k-1) + c) + d = (c+d)n + c. \blacksquare$$

Note the shape of that argument: the guess is *linear in  $n$*  and independent of the split  $k$ , which is exactly why the unbalanced split doesn't hurt — every node is visited once, and the recursion adds  $\Theta(1)$  per node.

**Why sorted order comes out:** by the BST property, everything in `x.left` is  $\leq x.key \leq$  everything in `x.right`. Recursion gives sorted order on each side, and printing `x` between them concatenates them correctly. This is the induction in one line.

Changing where `process(x)` sits relative to the two recursive calls gives **preorder** (first) and **postorder** (last). Skiena: these make little sense for search trees but are the right traversals when the tree encodes an arithmetic or logical expression.

## Querying: search, min, max, successor, predecessor

| TREE-SEARCH(x, k)                     | ITERATIVE-TREE-SEARCH(x, k)   |
|---------------------------------------|-------------------------------|
| 1 if x == NIL or k == x.key           | 1 while x ≠ NIL and k ≠ x.key |
| 2 return x                            | 2 if k < x.key                |
| 3 if k < x.key                        | 3 x = x.left                  |
| 4 return TREE-SEARCH(x.left, k)       | 4 else x = x.right            |
| 5 else return TREE-SEARCH(x.right, k) | 5 return x                    |

→ **C++ implementation:** [A2 TREE-SEARCH, ITERATIVE-TREE-SEARCH, TREE-MINIMUM, TREE-MAXIMUM](#)

Both follow a single **simple downward path** and cost  $O(h)$ . The iterative version is preferred on most machines — no call overhead, no recursion stack.

`TREE-MINIMUM` follows `left` until `NIL`; `TREE-MAXIMUM` follows `right`. Skiena's justification is worth internalizing because it is the BST property applied once: the smallest key must be in the left subtree of the root (everything left is smaller), and recursively, so it is the **left-most descendant** of the root.

```
TREE-SUCCESSOR(x)
1 if x.right ≠ NIL
2   return TREE-MINIMUM(x.right)           // case A: smallest
thing bigger than x
3 else                                     // case B: climb until
x is a LEFT child
4   y = x.p
5   while y ≠ NIL and x == y.right
6     x = y
7     y = y.p
8   return y
```

→ **C++ implementation:** [A3 TREE-SUCCESSOR](#)

**Why case B works.** If  $x$  has no right subtree, the successor is *the lowest ancestor of  $x$  whose left subtree contains  $x$* . Walking up while  $x$  is a right child skips every ancestor smaller than  $x$ ; the first time we arrive from the left, that ancestor is bigger than everything in its left subtree, i.e. bigger than  $x$ , and it is the smallest such. If we run off the root,  $x$  was the maximum and the successor is `NIL`.

**Theorem 12.2.** SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR each run in  $O(h)$  on a BST of height  $h$ .

## Insertion

```

TREE-INSERT(T, z)                                // z.key filled in; z.left
= z.right = NIL
1  x = T.root ;  y = NIL
2  while x ≠ NIL                                // descend, remembering the
    parent
3      y = x
4      if z.key < x.key  x = x.left  else  x = x.right
5  z.p = y
6  if y == NIL          T.root = z          // tree was empty
7  elseif z.key < y.key y.left  = z
8  else                 y.right = z

```

There is **exactly one** place a new key can go: the `NIL` pointer that an unsuccessful search for it would reach. Skiena states this as the design constraint — "we must replace the NULL pointer found in `T` after an unsuccessful query" — because that is the only spot where a later search will look.  $O(h)$  for the descent,  $O(1)$  for the splice.

Skiena's C version threads a `tree **l` (pointer to the parent's child-pointer) through the recursion so the assignment `*l = p` links the node in without a separate parent-tracking variable — a neat idiom, but the iterative CLRS version with an explicit trailing pointer `y` is easier to get right and is what you should write in an interview.

## Deletion — three shapes, four code cases

Deletion is where BST implementations go wrong. The conceptual case analysis has **three** shapes; CLRS's code has **four** branches because one shape splits on whether the successor is adjacent.

Skiena's three shapes (his Figure 3.5, worth drawing yourself):

1. **No children** — clear the parent's pointer.
2. **One child** — splice: link the child directly to the deleted node's parent. In-order labeling survives because the child subtree occupies exactly the same key range.
3. **Two children** — the hard one. Relabel the node with its **immediate successor** (the left-most descendant of the right subtree), then delete *that* node, which by construction has at most one child. Deletion is thereby reduced to case 1 or 2.

CLRS 4e implements this with a splice helper:

```

TRANSPLANT(T, u, v)                                // replace subtree u by
subtree v
1  if u.p == NIL      T.root = v
2  elseif u == u.p.left  u.p.left = v
3  else                u.p.right = v
4  if v ≠ NIL  v.p = u.p

```

→ C++ implementation: [A4 TRANSPLANT](#)

```

TREE-DELETE(T, z)
1  if z.left == NIL                                // case 1: 0 or 1 (right)
child
2      TRANSPLANT(T, z, z.right)
3  elseif z.right == NIL                          // case 2: exactly 1 (left)
child
4      TRANSPLANT(T, z, z.left)
5  else y = TREE-MINIMUM(z.right)                  // y = z's successor, has
no left child
6      if y ≠ z.right                              // case 4: y is deeper than
z.right
7          TRANSPLANT(T, y, y.right)              // lift y out of where it
lives
8          y.right = z.right                        // give y all of z's
right subtree
9          y.right.p = y
10     TRANSPLANT(T, z, y)                          // case 3 continues here: y
takes z's slot
11     y.left = z.left
12     y.left.p = y

```

→ C++ implementation: [A5 TREE-DELETE](#)

**Important 4e change.** Third edition *copied* `y`'s key into `z` and freed `y`. Fourth edition **moves the node `y` into `z`'s position and frees `z` itself**. The reason, given in the chapter notes: if other parts of the program hold pointers to tree nodes, copying keys silently makes an outside pointer to `z` now refer to a *different* key, while the pointer to `y` dangles. Moving nodes means the only invalidated pointer is the one to the node you actually asked to delete. **This is the same "handle problem" as in priority queues (M05).** Skiena's own text describes the relabeling version, and notes the full implementation "looks a little ghastly" — which it does; the `TRANSPLANT` factoring is what tames it.

**Theorem 12.3.** `TREE-DELETE` runs in  $O(h)$ . Everything is  $O(1)$  except the single call to `TREE-MINIMUM`.

## Common bugs in BST deletion

| BUG                                                                                                     | SYMPTOM                                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Forgetting <code>v.p = u.p</code> in <code>TRANSPLANT</code>                                            | Parent pointers rot; <code>SUCCESSOR</code> walks into garbage                                                                                          |
| Guarding <code>v.p = u.p</code> on <code>v ≠ NIL</code> and then relying on <code>x.p</code> afterwards | Works for plain BSTs, <b>breaks red-black delete</b> (which needs <code>NIL.p</code> set)                                                               |
| Using <code>predecessor</code> in one branch and <code>successor</code> in another without care         | Fine for correctness; but mixing them per-call is what keeps a BST from degenerating under alternating insert/delete — worth mentioning in an interview |
| Deleting by copying keys                                                                                | Invalidates external node handles (see 4e note above)                                                                                                   |
| Not handling <code>z == T.root</code>                                                                   | Null-deref on <code>z.p</code>                                                                                                                          |

## How good are binary search trees?

### Unified Understanding

All three dictionary operations cost  $O(h)$ . The best possible height is  $\lceil \lg(n+1) \rceil$  (perfectly balanced); the worst is  $n$ .

**The shape is decided by the insertion order, not by the data.** Skiena: the insertion algorithm puts each new item at the leaf where it *should have been found*, so the tree's shape is a function of the order the user hands you keys. Insert `a, b, c, d, ...` in sorted order and you get a "skinny, linear-height tree where only right pointers are used" — every operation degrades to  $\Theta(n)$ . **The data structure has no control over the order of**



**insertion**, and a hostile (or merely sorted) input stream is realistic: sorted files are the common case, not the rare one.

**Average case.** If all  $n!$  insertion orderings are equally likely, the resulting tree has height  $\Theta(\lg n)$  **with high probability**. Skiena flags this as "an important example of the power of randomization" and points to the same idea underlying quicksort (M05) — and indeed the analysis is the quicksort analysis: the root is a random pivot, and the recursion on subtree sizes is quicksort's recursion.

CLRS Problem 12-3 gets the companion result: the average **depth** of a node in a randomly built BST is  $O(\lg n)$  (this is  $P(n)/n$  where  $P(n)$  is the total path length, satisfying the quicksort recurrence).

### Outside / Engineering Context

*The sharp constant is due to Reed (2003): the expected height of a randomly built BST is  $\alpha \lg n - \beta \lg \lg n + O(1)$  with  $\alpha \approx 4.311$ , i.e. about  $2.99 \lg n$  asymptotically. Empirically, building a BST from a random permutation of 100 000 keys gives height  $\approx 40$  against  $\lg n \approx 16.6$  — a ratio of about 2.4, consistent with the  $-\beta \lg \lg n$  correction. So a random BST is a constant factor worse than a red-black tree, not an order of magnitude. This is why treaps and randomized BSTs are a legitimate engineering choice.*

**Counting the shapes** (CLRS Problem 12-4): the number of distinct binary trees on  $n$  nodes is the Catalan number

$$b_n = C(2n, n) / (n + 1) = 4^n / (\sqrt{\pi} \cdot n^{3/2}) \cdot (1 + O(1/n))$$

So there are exponentially many shapes but only  $n!$  insertion orders — many orders produce the same tree, and *balanced trees are produced by far more orders than degenerate ones*, which is the combinatorial reason random insertion tends to balance.

Recognition pattern — reach for a plain BST when...

- You need ordered operations (successor, range,  $k$ -th) that a hash table cannot give.
- Insertion order is genuinely random or you control it (e.g. you shuffle first).
- You're prototyping, or building a treap/randomized BST on purpose.

Otherwise: **use a balanced tree** — and in real C++ that means `std::map` / `std::set`.

## C++ Implementation — plain BST (CLRS 4e node-moving delete)

```
#include <vector>

template <class Key>
class BST {
public:
    struct Node {
        Key    key;
        Node *left = nullptr, *right = nullptr, *parent = nullptr;
        explicit Node(const Key& value) : key(value) {}
    };

    BST() = default;
    ~BST() { destroy(root_); }
    BST(const BST&) = delete;
    BST& operator=(const BST&) = delete;

    Node* root() const { return root_; }
    size_t size() const { return count_; }

    Node* find(const Key& key) const {
        Node* node = root_;
        while (node && node->key != key) node = (key < node->key) ?
node->left : node->right;
        return node;
    }

    static Node* minimum(Node* node) { while (node->left) node =
node->left; return node; }
    static Node* maximum(Node* node) { while (node->right) node =
node->right; return node; }

    static Node* successor(Node* node) {
        if (node->right) return minimum(node->right);
        Node* ancestor = node->parent;
        while (ancestor && node == ancestor->right) { node =
ancestor; ancestor = ancestor->parent; }
        return ancestor;
    }

    Node* insert(const Key& key) {
```

```

        Node* fresh = new Node(key);
        Node* parent = nullptr; // trails one step
        behind the walker
        Node* node = root_;
        while (node) { parent = node; node = (key < node->key) ?
node->left : node->right; }
        fresh->parent = parent;
        if (!parent) root_ = fresh;
        else if (key < parent->key) parent->left = fresh;
        else parent->right = fresh;
        ++count_;
        return fresh;
    }

    // Moves the successor node into target's position; frees
    target itself (CLRS 4e).
    void erase(Node* target) {
        if (!target->left) transplant(target, target-
>right);
        else if (!target->right) transplant(target, target->left);
        else {
            Node* replacement = minimum(target->right);
            if (replacement->parent != target) {
                transplant(replacement, replacement->right);
                replacement->right = target->right;
                replacement->right->parent = replacement;
            }
            transplant(target, replacement);
            replacement->left = target->left;
            replacement->left->parent = replacement;
        }
        delete target;
        --count_;
    }

    void inorder(vector<Key>& out) const { walk(root_, out); }

private:
    Node* root_ = nullptr;
    size_t count_ = 0;

```

```

    // Splices the subtree rooted at `newSubtree` into the slot
    `oldSubtree` occupies.
    void transplant(Node* oldSubtree, Node* newSubtree) {
        if (!oldSubtree->parent)                root_ =
newSubtree;
        else if (oldSubtree == oldSubtree->parent->left)
oldSubtree->parent->left = newSubtree;
        else
oldSubtree->parent->right = newSubtree;
        if (newSubtree) newSubtree->parent = oldSubtree->parent;
    }
    static void walk(Node* node, vector<Key>& out) {
        if (!node) return;
        walk(node->left, out);
        out.push_back(node->key);
        walk(node->right, out);
    }
    static void destroy(Node* node) {
        if (!node) return;
        destroy(node->left);
        destroy(node->right);
        delete node;
    }
};

```

*Verified:* 40 000 random interleaved insert/erase operations against `std::multiset` — inorder walk matches exactly, and the successor chain from `minimum` reproduces the same sorted sequence. Building from a random permutation of 100 000 keys gave height 40; inserting `0..1999` in sorted order gave height exactly 2000.

---

## Part 2 — Red-Black Trees (CLRS 13)

### Unified Understanding

A red-black tree is a BST with **one extra bit per node** (a color) and five constraints on how colors may be arranged. The constraints have exactly one purpose: they make it impossible for one root-to-leaf path to be more than twice as long as another, which forces  $h = O(\lg n)$ . All the machinery — the sentinel, rotations, the fixup cases — exists to restore those five properties in  $O(\lg n)$  time after an insert or delete.

Skiena's position on this is deliberately pragmatic and you should adopt it: *know that balanced trees exist, know their guarantees, and use them as black boxes*. "From an algorithm design viewpoint, it is important to know that these trees exist and that they can be used as black boxes to provide an efficient dictionary implementation. When figuring the costs of dictionary operations for algorithm analysis, we can assume the worst-case complexities of balanced binary trees to be a fair measure."

**CLRS emphasis:** *the full construction with proofs — you should be able to derive the height bound and explain each fixup case.*

**Skiena emphasis:** *you will almost never implement one; you must be able to cost one instantly and pick it correctly.*

## The five properties

1. Every node is either **red** or **black**.
2. The **root** is black.
3. Every **leaf** (`T.nil`) is black.
4. If a node is red, then **both its children are black**. (No two reds in a row on any path.)
5. For each node, all simple paths from that node down to descendant leaves contain the **same number of black nodes**.

Properties 4 and 5 are the whole game: 5 says paths agree on black count; 4 says reds cannot pad a path by more than doubling it.

**The sentinel `T.nil`.** All `NIL` pointers point to one shared black sentinel node, whose `key`, `left`, `right` are unused (`p` is used and gets scribbled on deliberately). This is the sentinel trick from M06: it removes every boundary test from the fixup code, which otherwise would be a thicket of null checks. One sentinel per tree, not one per leaf.

**Black-height.** `bh(x)` = the number of black nodes on any simple path from `x` down to a leaf, **not counting `x` itself but counting the leaf**. Property 5 makes this well-defined.  
`bh(T) = bh(T.root)`.

## Lemma 13.1 — the height bound

**Lemma 13.1.** A red-black tree with `n` internal nodes has height  $h \leq 2 \lg(n + 1)$ .

*Proof skeleton (two steps — memorize this shape).*

**Step 1 (induction on height):** the subtree rooted at any node  $x$  contains at least  $2^{\text{bh}(x)} - 1$  internal nodes.

- *Base:*  $h(x) = 0 \Rightarrow x$  is `T.nil`,  $\text{bh}(x) = 0$ , and  $2^0 - 1 = 0$ . ✓
- *Step:*  $x$  has two children, each of height  $\leq h(x) - 1$ . Each child has black-height  $\text{bh}(x)$  (if the child is red) or  $\text{bh}(x) - 1$  (if black) — in either case  $\geq \text{bh}(x) - 1$ . By induction each child's subtree has  $\geq 2^{\text{bh}(x)-1} - 1$  nodes, so  $x$ 's subtree has  $\geq 2(2^{\text{bh}(x)-1} - 1) + 1 = 2^{\text{bh}(x)} - 1$ . ✓

**Step 2 (property 4 relates  $\text{bh}$  to  $h$ ):** at least half the nodes on any root-to-leaf simple path (excluding the root) must be black — because reds can't be adjacent. Hence  $\text{bh}(\text{root}) \geq h/2$ .

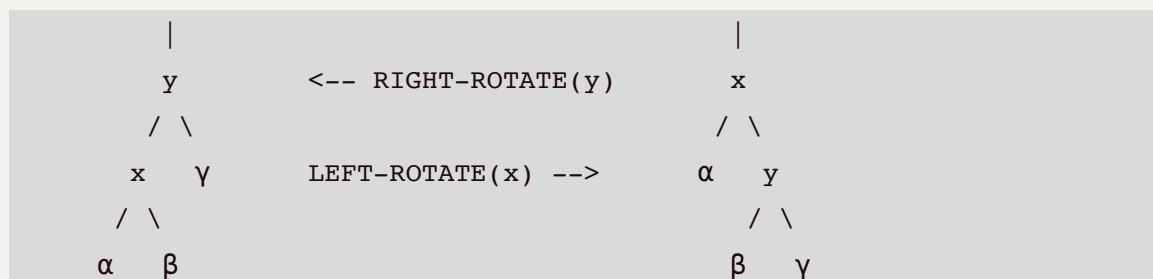
**Combine:**  $n \geq 2^{\lceil h/2 \rceil} - 1$ , so  $n + 1 \geq 2^{\lceil h/2 \rceil}$ , so  $\lg(n+1) \geq h/2$ , so  $h \leq 2 \lg(n+1)$ . ■

**Immediate corollary:** SEARCH, MINIMUM, MAXIMUM, SUCCESSOR, PREDECESSOR all run in  $O(\lg n)$  on a red-black tree — those are the plain BST algorithms, unchanged, riding on a guaranteed-shallow tree.

*Verified:* on a red-black tree holding 5505 keys after 60 000 random operations, measured height was 16 against the bound  $2 \lg(n+1) = 24.85$ .

## Rotations

INSERT and DELETE modify the tree and can break properties 2 or 4. Recoloring alone isn't enough; you also need to change **shape** without changing the **inorder order**. That is exactly what a rotation does.



Read the inorder sequence off both sides:  $\alpha \ x \ \beta \ \gamma \ \gamma$ . Identical. That is the correctness proof of a rotation in one line.

```

LEFT-ROTATE(T, x)                                // assumes x.right ≠
T.nil
1  y = x.right
2  x.right = y.left                                // y's left subtree
   becomes x's right
3  if y.left ≠ T.nil  y.left.p = x
4  y.p = x.p                                       // link y to x's parent
5  if x.p == T.nil      T.root = y
6  elseif x == x.p.left  x.p.left = y
7  else                  x.p.right = y
8  y.left = x                                                  // put x on y's left
9  x.p = y

```

→ **C++ implementation:** [A6 LEFT-ROTATE and RIGHT-ROTATE](#)

$O(1)$  — a fixed number of pointer writes. `RIGHT-ROTATE` is the mirror image. **Both change only pointers, never keys.**

## Insertion

`RB-INSERT` is `TREE-INSERT` with `T.nil` in place of `NIL`, plus: color the new node **red**, then call `RB-INSERT-FIXUP`.

**Why red?** (Exercise 13.3-1, and the key intuition.) Inserting a black node would immediately break property 5 on every path through it — a global violation, hard to repair. Inserting a red node preserves property 5 for free and can only break property 4 (a red child of a red parent) or property 2 (if the tree was empty and `z` is the root). Both are *local* and repairable.

```

RB-INSERT-FIXUP(T, z)
1  while z.p.color == RED
2      if z.p == z.p.p.left
3          y = z.p.p.right                // y = z's UNCLE
4          if y.color == RED              // ---- Case
1
5              z.p.color = BLACK
6              y.color = BLACK
7              z.p.p.color = RED          // push blackness
down
8              z = z.p.p                  // move violation up
TWO levels

```

```

9         else
10             if z == z.p.right                // ---- Case
11                 z = z.p
12                 LEFT-ROTATE(T, z)            // turn into Case 3
13                 z.p.color = BLACK            // ---- Case
14                 z.p.p.color = RED
15                 RIGHT-ROTATE(T, z.p.p)        // loop will now exit
16     else (same with "right" and "left" exchanged)
17     T.root.color = BLACK

```

→ C++ implementation: [A7 RB-INSERT-FIXUP](#)

The three cases, by the color of the uncle **y**:

| CASE | CONDITION                                                                       | ACTION                                                                     | EFFECT                                                                                                                            |
|------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 1    | uncle <b>y</b> is <b>red</b>                                                    | recolor: parent &<br>uncle → black,<br>grandparent → red                   | grandparent's blackness moves <i>down</i><br>one level to both children; violation<br>moves <b>up two levels</b> ; loop continues |
| 2    | uncle black, <b>z</b> is<br>a <b>right</b> child<br>(parent is a left<br>child) | LEFT-ROTATE(z.p)                                                           | converts to case 3; no colors change;<br><b>z</b> moves down one, parent up one                                                   |
| 3    | uncle black, <b>z</b> is<br>a <b>left</b> child                                 | recolor parent black,<br>grandparent red,<br>RIGHT-<br>ROTATE(grandparent) | fixes the violation outright; <b>loop<br/>terminates</b>                                                                          |

Note cases 2 and 3 are not mutually exclusive with each other in the flow — case 2 *falls into* case 3.

**Loop invariant (three parts) — worth memorizing because it is a model for how to state one:**

At the start of each iteration of the while loop:

- (a) node **z** is red.
- (b) if **z.p** is the root, then **z.p** is black.
- (c) if the tree violates any red-black property, it violates **at most one** of them,



and the violation is *either* property 2 (because  $z$  is a red root) *or* property 4 (because  $z$  and  $z.p$  are both red).

*Initialization.*  $z$  is the newly inserted red node  $\Rightarrow$  (a). If  $z.p$  is the root it was black before insertion and insertion didn't change it  $\Rightarrow$  (b). Property 1 and 3 hold trivially; property 5 holds because  $z$  replaced a black  $T.nil$  with a red node having two black  $T.nil$  children — same black count. If property 2 is violated,  $z$  is a red root and no other violation exists (the tree had only one node). If property 4 is violated, it's between  $z$  and  $z.p$ , and nothing else, because everything else was legal  $\Rightarrow$  (c).

*Maintenance, case 1.* Let  $z' = z.p.p$  be the next  $z$ . (a) case 1 colors  $z.p.p$  red, so  $z'$  is red. (b)  $z'.p$  is unchanged in color; if it's the root it was black and stays black. (c) property 5 is preserved because the blackness moved *down* symmetrically to both children — every downward path still has the same black count; properties 1 and 3 untouched. If  $z'$  is now the root, the only possible violation is property 2, due to  $z'$ . If not: case 1 fixed the one property-4 violation, made  $z'$  red, and left  $z'.p$  alone — so either  $z'.p$  was black (no violation) or was red (exactly one new property-4 violation, between  $z'$  and  $z'.p$ ).

*Maintenance, cases 2 and 3.* (a) case 2 makes  $z$  point to  $z.p$ , which is red; nothing else recolors  $z$ . (b) case 3 makes  $z.p$  black, so if it becomes the root it's black. (c) rotations in cases 2 and 3 preserve property 5; case 3 makes a node red only as a *child of a black node* after the rotation, so property 2 can't break; and the single property-4 violation is corrected without introducing another.

*Termination.* If only case 1 runs,  $z$  climbs two levels per iteration and eventually  $z.p$  is black (or  $z$  is the root and  $z.p == T.nil$ , which is black). Cases 2/3 terminate the loop directly. On exit, property 4 holds (that's the loop condition failing); by the invariant, the only property that might fail is property 2, and line 17 fixes it by blackening the root. ■

**Complexity.**  $O(\lg n)$  for the descent. In fixup, the loop repeats **only when case 1 occurs**, and case 1 moves  $z$  up two levels — so at most  $O(\lg n)$  iterations. Cases 2 and 3 each terminate the loop, so **RB-INSERT performs at most 2 rotations**. Total  $O(\lg n)$ .

*Why "at most 2 rotations" matters: it is a load-bearing fact for augmentation (Theorem 17.1). If a rebalancing scheme could do  $\Theta(\lg n)$  rotations per operation and each rotation forced an update path to the root, a single operation would cost  $\Theta(\lg^2 n)$ . See §17 below and the chapter notes on left-leaning red-black trees, which are shorter to code but **do not** bound rotations per operation by a constant.*

## Deletion

Deletion is genuinely harder. `RB-DELETE` is `TREE-DELETE` with three additions: `RB-TRANSPLANT`, tracking `y`'s original color, and the fixup call.

```
RB-TRANSPLANT(T, u, v)
1  if u.p == T.nil      T.root = v
2  elseif u == u.p.left  u.p.left  = v
3  else                  u.p.right = v
4  v.p = u.p             // UNCONDITIONAL - v may be
                          T.nil, and we need its parent
```

→ C++ implementation: [A8 RB-TRANSPLANT](#)

That unconditional line 4 is the difference from `TRANSPLANT`, and it is not cosmetic: `RB-DELETE-FIXUP` reads `x.p` repeatedly, and `x` may be the sentinel.

```
RB-DELETE(T, z)
1  y = z
2  y-original-color = y.color
3  if z.left == T.nil
4      x = z.right ; RB-TRANSPLANT(T, z, z.right)
5  elseif z.right == T.nil
6      x = z.left  ; RB-TRANSPLANT(T, z, z.left)
7  else y = TREE-MINIMUM(z.right)           // y is z's successor
8      y-original-color = y.color
9      x = y.right
10     if y ≠ z.right                        // y is farther down
11         RB-TRANSPLANT(T, y, y.right)
12         y.right = z.right ; y.right.p = y
13     else x.p = y                         // in case x is T.nil
14     RB-TRANSPLANT(T, z, y)
15     y.left = z.left ; y.left.p = y
16     y.color = z.color                    // y inherits z's
color
17  if y-original-color == BLACK
18      RB-DELETE-FIXUP(T, x)
```

→ C++ implementation: [A9 RB-DELETE](#)

The reasoning, compressed:

- `y` is the node **physically removed or moved**. If `z` has  $\leq 1$  child, `y = z`. If `z` has

2 children,  $y$  is  $z$ 's successor, which has no left child, and it moves into  $z$ 's position **taking  $z$ 's color** (line 16).

- $x$  is the node that **moves into  $y$ 's original position** —  $y$ 's only child, or  $T.nil$ .
- **If  $y$  was red, nothing breaks.** Three reasons: (1) no black-heights change (a red node contributes 0 to every path count); (2) no two reds become adjacent — if  $y = z$  its parent and children were black, and if  $y$  moved into  $z$ 's slot it wears  $z$ 's color there, while  $y$ 's old right child  $x$  must be black since  $y$  was red; (3)  $y$  red  $\Rightarrow y$  wasn't the root  $\Rightarrow$  the root stays black.
- **If  $y$  was black, three things can break:** property 2 (if  $y$  was the root and a red child moved up), property 4 (if  $x$  and its new parent are both red), and property 5 (every path that went through  $y$  lost a black).

**The "extra black" device.** To repair property 5, imagine that when black  $y$  is removed, its blackness **transfers to  $x$** . Now  $x$  counts as 2 blacks (if it was black — "doubly black") or 1 (if it was red — "red-and-black"). Property 5 is restored *by fiat*; the cost is that property 1 is now violated ( $x$  is neither purely red nor purely black). **The extra black lives in the pointer  $x$ , not in the color attribute.** `RB-DELETE-FIXUP`'s job is to push that extra black up or out of the tree.

The loop ends when one of three things happens:

1.  $x$  is **red-and-black**  $\rightarrow$  just color it black (line 44). The extra black is absorbed.
2.  $x$  is the **root**  $\rightarrow$  the extra black simply vanishes (removing a black from *every* path is harmless).
3. Suitable rotations and recolorings eliminate it locally.

```
RB-DELETE-FIXUP(T, x)                                // shown for x a LEFT
child; mirror otherwise
1  while x  $\neq$  T.root and x.color == BLACK
2      if x == x.p.left
3          w = x.p.right                                // w = x's SIBLING; w  $\neq$ 
T.nil always
4          if w.color == RED                                //
---- Case 1
5              w.color = BLACK ; x.p.color = RED
7              LEFT-ROTATE(T, x.p) ; w = x.p.right
9              if w.left.color == BLACK and w.right.color == BLACK
// ---- Case 2
10             w.color = RED ; x = x.p
//  push extra black up
```

```

12         else
13             if w.right.color == BLACK
14 // ---- Case 3
15                 w.left.color = BLACK ; w.color = RED
16                 RIGHT-ROTATE(T, w) ; w = x.p.right
17                 w.color = x.p.color
18 // ---- Case 4
19                 x.p.color = BLACK ; w.right.color = BLACK
20                 LEFT-ROTATE(T, x.p) ; x = T.root
21 // done
44 x.color = BLACK

```

→ C++ implementation: [A10 RB-DELETE-FIXUP](#)

**Why  $w \neq T.nil$ :**  $x$  is doubly black, so the path from  $x.p$  through  $x$  already counts an extra black; if the sibling were the sentinel, the path from  $x.p$  to that (singly black) leaf would have strictly fewer blacks — contradicting property 5.

| CASE | CONDITION<br>(SIBLING $w$ )              | ACTION                                                                                                              | OUTCOME                                                                                                                                                                                                     |
|------|------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | $w$ is red                               | swap colors of $w$ and $x.p$ , LEFT-ROTATE( $x.p$ )                                                                 | $w$ red $\Rightarrow$ its children are black $\Rightarrow$ the new sibling is black. <b>Converts to case 2, 3, or 4.</b>                                                                                    |
| 2    | $w$ black, both $w$ 's children black    | color $w$ red, $x = x.p$                                                                                            | removes one black from $x$ and from $w$ , compensating by giving $x.p$ the extra black. <b>Only looping case.</b> If reached via case 1, the new $x$ is red-and-black $\Rightarrow$ loop exits immediately. |
| 3    | $w$ black, $w.left$ red, $w.right$ black | swap colors of $w$ and $w.left$ , RIGHT-ROTATE( $w$ )                                                               | new sibling is black with a <b>red right child</b> . <b>Falls through to case 4.</b>                                                                                                                        |
| 4    | $w$ black, $w.right$ red                 | $w.color = x.p.color$ ;<br>$x.p.color = BLACK$ ;<br>$w.right.color = BLACK$ ;<br>LEFT-ROTATE( $x.p$ ); $x = T.root$ | the extra black vanishes without breaking anything. <b>Terminates.</b>                                                                                                                                      |

**Why each transformation is legal:** in every case the number of black nodes (counting  $x$ 's extra black) from the root of the shown subtree down to the root of each of the subtrees  $\alpha$ ,  $\beta$ , ...,  $\zeta$  is unchanged. Counting with `count(RED) = 0`, `count(BLACK) = 1` and carrying the parent's unknown color  $c$  symbolically makes this a two-line verification per case (CLRS Exercise 13.4-6).

**Complexity.** The `RB-DELETE` body without fixup is  $O(\lg n)$ . In fixup, cases 1, 3, 4 each terminate after  $O(1)$  work and at most 3 rotations total; case 2 is the only looping case and it moves  $x$  up one level with **no rotations**, at most  $O(\lg n)$  times. So `RB-DELETE` is  $O(\lg n)$  with **at most 3 rotations**.

## Insert vs delete at a glance

|                      | INSERT                                        | DELETE                                                     |
|----------------------|-----------------------------------------------|------------------------------------------------------------|
| Violation introduced | property 4 (red-red), or property 2           | property 5 (missing black), plus possibly 2 and 4          |
| Repair device        | recolor + rotate; the violation <i>climbs</i> | "extra black" on a pointer; the extra black <i>climbs</i>  |
| Looping case         | case 1 (uncle red), climbs 2 levels           | case 2 (sibling black, both nephews black), climbs 1 level |
| Max rotations        | 2                                             | 3                                                          |
| Cases                | 3 ( $\times 2$ mirrored)                      | 4 ( $\times 2$ mirrored)                                   |

## C++ Implementation — red-black tree with a pluggable augmentation

This one implementation serves the rest of the module. The template parameter `Aug` is a small struct carrying the extra attribute and a `pull` function that recomputes it from the node and its two children — which is **exactly the hypothesis of Theorem 17.1** expressed in the type system. Swap `Aug` and you get an order-statistic tree or an interval tree with no change to the balancing code.

```
#include <algorithm>
#include <climits>
#include <cstdint>
#include <functional>

template <class Aug, class Compare = less<typename Aug::Key>>
class RBTree {
```

```

public:
    using Key = typename Aug::Key;

    struct Node {
        Key    key{};
        Node *left, *right, *parent;
        bool   red;
        Aug    aug{};
    };

    RBTree() {
        nil_ = new Node{Key{}, nullptr, nullptr, nullptr, false,
Aug{}};
        nil_>left = nil_>right = nil_>parent = nil_;
        root_ = nil_;
    }
    ~RBTree() { destroy(root_); delete nil_; }
    RBTree(const RBTree&) = delete;
    RBTree& operator=(const RBTree&) = delete;

    Node* nil() const { return nil_; }
    Node* root() const { return root_; }
    size_t size() const { return count_; }

    Node* find(const Key& key) const {
        Node* node = root_;
        while (node != nil_) {
            if (less_(key, node->key))      node = node->left;
            else if (less_(node->key, key)) node = node->right;
            else                            return node;
        }
        return nil_;
    }

    Node* minimum(Node* node) const { while (node->left != nil_)
node = node->left; return node; }

    Node* successor(Node* node) const {
        if (node->right != nil_) return minimum(node->right);
        Node* ancestor = node->parent;
        while (ancestor != nil_ && node == ancestor->right) { node
= ancestor; ancestor = ancestor->parent; }
    }

```

```

        return ancestor;
    }

    Node* insert(const Key& key) {
        Node* fresh = new Node{key, nil_, nil_, nil_, true,
Aug{}};
        Node* parent = nil_; // trails one step
behind the walker
        Node* node = root_;
        while (node != nil_) { parent = node; node = less_(key,
node->key) ? node->left : node->right; }
        fresh->parent = parent;
        if (parent == nil_) root_ = fresh;
        else if (less_(key, parent->key)) parent->left = fresh;
        else parent->right = fresh;
        pullUp(fresh); // fresh and every proper
ancestor
        insertFixup(fresh);
        ++count_;
        return fresh;
    }

    void erase(Node* target) {
        Node* removed = target; // the node physically
unlinked or moved
        Node* fixupAt; // node that slides into
`removed`'s old slot
        Node* recomputeFrom; // deepest node whose
subtree changed
        bool removedWasBlack = !removed->red;
        if (target->left == nil_) {
            fixupAt = target->right; transplant(target, target-
>right); recomputeFrom = target->parent;
        } else if (target->right == nil_) {
            fixupAt = target->left; transplant(target, target-
>left); recomputeFrom = target->parent;
        } else {
            removed = minimum(target->right);
            removedWasBlack = !removed->red;
            fixupAt = removed->right;
            if (removed->parent == target) { fixupAt->parent =
removed; recomputeFrom = removed; }

```

```

        else {
            transplant(removed, removed->right);
            recomputeFrom = removed->parent;
            removed->right = target->right;
            removed->right->parent = removed;
        }
        transplant(target, removed);
        removed->left = target->left;
        removed->left->parent = removed;
        removed->red = target->red;
    }
    pullUp(recomputeFrom);
    if (removedWasBlack) deleteFixup(fixupAt);
    delete target;
    --count_;
}

private:
    Node* root_;
    Node* nil_;
    size_t count_ = 0;
    Compare less_{};

    void pull(Node* node) { if (node != nil_) node-
>aug.pull(node); }
    void pullUp(Node* node) { while (node != nil_) { node-
>aug.pull(node); node = node->parent; } }

    // `pivot` descends one level; its right child `newParent`
    takes its place.
    void leftRotate(Node* pivot) {
        Node* newParent = pivot->right;
        pivot->right = newParent->left;
        if (newParent->left != nil_) newParent->left->parent =
pivot;
        newParent->parent = pivot->parent;
        if (pivot->parent == nil_)            root_ = newParent;
        else if (pivot == pivot->parent->left) pivot->parent->left
= newParent;
        else                                pivot->parent-
>right = newParent;
        newParent->left = pivot;
    }

```



```

        pivot->parent = newParent;
        pull(pivot); pull(newParent);    // only these two subtrees
changed
    }

    void rightRotate(Node* pivot) {
        Node* newParent = pivot->left;
        pivot->left = newParent->right;
        if (newParent->right != nil_) newParent->right->parent =
pivot;
        newParent->parent = pivot->parent;
        if (pivot->parent == nil_)            root_ = newParent;
        else if (pivot == pivot->parent->right) pivot->parent-
>right = newParent;
        else                                pivot->parent->left
= newParent;
        newParent->right = pivot;
        pivot->parent = newParent;
        pull(pivot); pull(newParent);
    }

    void transplant(Node* oldSubtree, Node* newSubtree) {
        if (oldSubtree->parent == nil_)            root_ =
newSubtree;
        else if (oldSubtree == oldSubtree->parent->left)
oldSubtree->parent->left = newSubtree;
        else
oldSubtree->parent->right = newSubtree;
        newSubtree->parent = oldSubtree->parent;    //
unconditional: newSubtree may be the sentinel
    }

    void insertFixup(Node* node) {
        while (node->parent->red) {
            if (node->parent == node->parent->parent->left) {
                Node* uncle = node->parent->parent->right;
                if (uncle->red) {                                // case
1
                    node->parent->red = false; uncle->red = false;
                    node->parent->parent->red = true;
                    node = node->parent->parent;
                } else {

```

```

2         if (node == node->parent->right) {           // case

            node = node->parent;
            leftRotate(node);
        }
        node->parent->red = false;                       // case
3
        node->parent->parent->red = true;
        rightRotate(node->parent->parent);
    }
} else {
    Node* uncle = node->parent->parent->left;
    if (uncle->red) {
        node->parent->red = false; uncle->red = false;
        node->parent->parent->red = true;
        node = node->parent->parent;
    } else {
        if (node == node->parent->left) { node = node-
>parent; rightRotate(node); }
        node->parent->red = false;
        node->parent->parent->red = true;
        leftRotate(node->parent->parent);
    }
}
}
root_>red = false;
}

void deleteFixup(Node* node) {
    while (node != root_ && !node->red) {
        if (node == node->parent->left) {
            Node* sibling = node->parent->right;
            if (sibling->red) {
                // case 1
                sibling->red = false; node->parent->red = true;
                leftRotate(node->parent);
                sibling = node->parent->right;
            }
            if (!sibling->left->red && !sibling->right->red) {
                // case 2
                sibling->red = true;
                node = node->parent;
            }
        }
    }
}

```

```

    } else {
        if (!sibling->right->red) {
// case 3
            sibling->left->red = false;
            sibling->red = true;
            rightRotate(sibling);
            sibling = node->parent->right;
        }
        sibling->red = node->parent->red;
// case 4
        node->parent->red = false;
        sibling->right->red = false;
        leftRotate(node->parent);
        node = root_;
    }
} else {
    Node* sibling = node->parent->left;
    if (sibling->red) {
        sibling->red = false; node->parent->red = true;
        rightRotate(node->parent);
        sibling = node->parent->left;
    }
    if (!sibling->right->red && !sibling->left->red) {
sibling->red = true; node = node->parent; }
    else {
        if (!sibling->left->red) {
            sibling->right->red = false; sibling->red =
true;

            leftRotate(sibling);
            sibling = node->parent->left;
        }
        sibling->red = node->parent->red;
        node->parent->red = false;
        sibling->left->red = false;
        rightRotate(node->parent);
        node = root_;
    }
}
}
node->red = false;
}

```

```

void destroy(Node* node) {
    if (node == nil_) return;
    destroy(node->left);
    destroy(node->right);
    delete node;
}
};

```

## Implementation notes.

- `pullUp(fresh)` after insertion walks from the new node to the root,  $O(\lg n)$  — that is the generic version of "increment sizes on the way down". The rotations inside `insertFixup` then repair only the two nodes they touch, because a rotation changes no other subtree's *contents*.
- On erase, `recomputeFrom` is deliberately chosen as the **deepest node whose subtree contents changed**, in each of the three structural shapes. In the "successor is far away" shape, `recomputeFrom = removed->parent` is a node that ends up *inside* `removed`'s new right subtree, so walking up from it passes through `removed` and then to the root — exactly the set of nodes needing recomputation.
- `transplant` assigns `newSubtree->parent` unconditionally. Removing that condition is the whole reason `deleteFixup` can read `node->parent` when `node` is the sentinel.
- The sentinel's `aug` is left at its default (`size = 0`, `maxHigh = LLONG_MIN`) and `pull` is never called on it — those defaults are the **identity elements** of the two augmentations.

*Verified:* 60 000 random interleaved insert/erase operations, with full red-black property checking (root black, no red-red, equal black-heights on every path, parent pointers consistent) every 997 steps and at the end; inorder walk matched `std::multiset` exactly; measured height 16 vs. the  $2 \lg(n+1) = 24.85$  bound.

## Common bugs in red-black code

| BUG                                                                         | SYMPTOM                                                                                                                                                             |
|-----------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Making <code>TRANSPLANT</code> 's parent assignment conditional             | <code>deleteFixup</code> reads a stale <code>x-&gt;p</code> ; corruption only on deletes where <code>x</code> is the sentinel — rare, so tests pass for a long time |
| Forgetting <code>T.root.color = BLACK</code> at the end of insert fixup     | Property 2 violated; height bound proof no longer applies                                                                                                           |
| Using a <i>fresh</i> nil node per leaf                                      | Wastes memory and <code>x-&gt;p</code> tricks stop working                                                                                                          |
| Not resetting <code>T.nil.color</code> to black after fixup scribbles on it | Infinite loop in the next fixup                                                                                                                                     |
| Recomputing augmented data only at the insertion point, not up to the root  | Sizes/maxima silently wrong; queries return plausible-but-wrong answers                                                                                             |
| Recomputing augmented data on the whole path <i>after</i> every rotation    | Correct but $\Theta(\lg^2 n)$ — rotations must be $O(1)$ to update                                                                                                  |

Other balanced schemes (chapter notes — know the names)

| SCHEME                                              | BALANCE DEVICE                               | NOTE                                                                                                                                                               |
|-----------------------------------------------------|----------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>AVL trees</b> (1962, Adel'son-Vel'skiĭ & Landis) | height difference of siblings $\leq 1$       | An AVL tree of height $h$ has $\geq F_h$ nodes $\Rightarrow h = O(\lg n)$ ; <b>stricter balance than RB</b> $\Rightarrow$ faster lookups, more rotations on update |
| <b>2-3 trees</b> (Hopcroft 1970)                    | node degree 2 or 3                           | Precursor to B-trees; a red-black tree <i>is</i> a 2-3-4 tree with each 3- or 4-node expanded into black+red nodes                                                 |
| <b>B-trees</b> (Bayer & McCreight 1972)             | node degree $\geq 2$                         | Part 4 of this module                                                                                                                                              |
| <b>AA-trees</b> (Andersson)                         | RB variant where left children are never red | Simpler to code                                                                                                                                                    |
| <b>Left-leaning red-black</b> (Sedgewick & Wayne)   | only left children may be red                | Much shorter code, <b>but does not bound rotations per operation by a constant</b> — which breaks the $O(\lg n)$ augmentation guarantee                            |
| <b>Treaps</b> (Seidel & Aragon)                     | BST on keys, heap on random priorities       | $O(\lg n)$ expected, very short code; LEDA's default dictionary                                                                                                    |
| <b>Splay trees</b> (Sleator & Tarjan)               | rotate accessed node to the root             | No balance condition at all; $O(\lg n)$ <b>amortized</b> (see M09); self-adjusting, great for skewed access patterns                                               |
| <b>Skip lists</b> (Pugh)                            | randomized tower of linked lists             | $O(\lg n)$ expected; often easier to make lock-free                                                                                                                |
| <b>Scapegoat / weight-balanced</b>                  | rebuild a subtree when it gets lopsided      | No per-node balance bits                                                                                                                                           |

Skiena adds **splay trees** and red-black trees as the two he singles out for implementation discussion.

---

## Part 3 — Augmenting Data Structures (CLRS 17.1–17.3)

## The four-step method

### Unified Understanding

Real problems rarely want a bare dictionary; they want a dictionary that also answers one extra kind of question. Rather than inventing a structure from scratch, **take a textbook structure and hang extra fields on it**. CLRS turns this into a procedure you can run mechanically:

1. **Choose an underlying data structure.** (Usually a red-black tree.)
2. **Determine additional information to maintain** in that structure.
3. **Verify that the additional information can be maintained** by the existing modifying operations, without hurting their asymptotic cost.
4. **Develop new operations** that use the information.

These steps are a guide, not a ritual — in practice you iterate, because step 4 often tells you the field you picked in step 2 was the wrong one.

**Theorem 17.1 (Augmenting a red-black tree).** Let  $f$  be an attribute augmenting a red-black tree  $T$  of  $n$  nodes, and suppose the value of  $f$  for each node  $x$  depends **only** on the information in nodes  $x$ ,  $x.\text{left}$ ,  $x.\text{right}$  (possibly including  $x.\text{left}.f$  and  $x.\text{right}.f$ ), and that  $x.f$  can be computed from this information in  $O(1)$  time. Then insertion and deletion can maintain the values of  $f$  in all nodes of  $T$  **without asymptotically affecting the  $O(\lg n)$  running times** of those operations.

*Proof skeleton.* A change to  $x.f$  propagates **only to ancestors** of  $x$  — no other node's  $f$  depends on  $x$ . Insertion's structural phase adds a leaf and updates  $O(\lg n)$  ancestors at  $O(1)$  each. Deletion's structural phase likewise touches one root-to-node path. Then the rebalancing phase: each rotation changes the children of exactly two nodes, so  $f$  can be recomputed for those two in  $O(1)$ , and only their ancestors are affected — which are already on the update path. **The crucial ingredient is that RB insert and delete perform  $O(1)$  rotations.** If a scheme performed  $\Theta(\lg n)$  rotations and each needed an update walk to the root, one operation would cost  $\Theta(\lg^2 n)$ . ■

Say that last sentence out loud in an interview and you will sound like you have actually read the chapter.

*Outside / Engineering Context*

This is the same idea as a **segment tree** or a **Fenwick tree**: a tree where each internal node caches an associative fold of its subtree. The `pull` function in the C++ code above is the fold; `Aug`'s default-constructed value is the identity. Exercise 17.2-3 makes this explicit: for any associative operator  $\otimes$  and per-node attribute `a`, the fold `x.f =  $x_1.a \otimes x_2.a \otimes \dots \otimes x_m.a$`  over the inorder listing of `x`'s subtree can be updated in  $O(1)$  after a rotation. Sum, min, max, gcd, matrix product, "count of elements satisfying a static predicate" all qualify. A **non-associative or non-invertible aggregate** is exactly the case where this fails.

## Order-statistic trees (§17.1)

**Problem.** Support, in addition to the dictionary operations, `SELECT(i)` (the `i`-th smallest key) and `RANK(x)` (the position of node `x` in the sorted order) in  $O(\lg n)$ .

**Augmentation.** Each node stores `x.size` = the number of **internal** nodes in the subtree rooted at `x`, including `x`:

```
x.size = x.left.size + x.right.size + 1      with   T.nil.size =
0
```

Setting the sentinel's size to 0 removes every boundary case — the same sentinel trick again.

```
OS-SELECT(x, i)                                // i-th smallest in x's
subtree, 1-indexed
1  r = x.left.size + 1                          // rank of x within its
own subtree
2  if i == r    return x
3  elseif i < r return OS-SELECT(x.left, i)
4  else        return OS-SELECT(x.right, i - r)
```

→ **C++ implementation:** [A11 OS-SELECT](#)

*Intuition:* `x.left.size` elements sit below `x`, so `x` itself is the `r`-th. If we want something smaller, recurse left with the same `i`; if larger, recurse right after **subtracting the `r` elements we skipped**. That subtraction is the whole trick and the most common place to be off by one.



```

OS-RANK(T, x)                                // position of x in the
sorted order
1  r = x.left.size + 1                        // x's rank within its own
subtree
2  y = x
3  while y ≠ T.root
4      if y == y.p.right                      // y's whole left sibling
subtree + parent precede x
5          r = r + y.p.left.size + 1
6      y = y.p
7  return r

```

→ C++ implementation: [A12 OS-RANK](#)

**Loop invariant:** at the start of each iteration,  $r$  is the rank of  $x.key$  in the subtree rooted at  $y$ .

- *Initialization:*  $r = x.left.size + 1$  is  $x$ 's rank in  $x$ 's own subtree;  $y = x$ . ✓
- *Maintenance:* if  $y$  is a **left** child, nothing in  $y.p$ 's left-side additions precedes  $x$  beyond what's counted —  $r$  is unchanged and is now  $x$ 's rank in  $y.p$ 's subtree. If  $y$  is a **right** child, then  $y.p$  and all of  $y.p.left$ 's subtree precede  $x$ , so add  $y.p.left.size + 1$ . ✓
- *Termination:*  $y = T.root$ , so  $r$  is  $x$ 's rank in the whole tree. ✓

Both are  $O(\lg n)$ : OS-SELECT descends one path, OS-RANK ascends one path.

**Maintaining sizes.** Two places.

- **Insertion:** on the downward pass, increment  $x.size$  for every node visited (the new key lands in every one of those subtrees). Then the fixup's rotations must repair sizes.
- **Rotation:** add exactly two lines to LEFT-ROTATE, after the pointer surgery:

```

y.size = x.size                                // y now roots what
x used to root
x.size = x.left.size + x.right.size + 1 // recompute x
bottom-up

```

$O(1)$ , and no other node's size changes because the rotation only permutes nodes *within* one subtree.

- **Deletion:** decrement sizes along the path from the physically removed node up to the root, then repair after the  $\leq 3$  rotations.

## Interval trees (§17.3)

**Problem.** Maintain a dynamic set of **closed intervals**  $i = [i.\text{low}, i.\text{high}]$ , supporting `INTERVAL-INSERT`, `INTERVAL-DELETE`, and `INTERVAL-SEARCH(i)` — return *some* interval in the set that overlaps  $i$ , or `T.nil` if none does.

**Interval trichotomy.** For any two intervals  $i$  and  $i'$ , exactly one holds:

1.  $i$  and  $i'$  overlap,
2.  $i$  is entirely to the left:  $i.\text{high} < i'.\text{low}$ ,
3.  $i$  is entirely to the right:  $i'.\text{high} < i.\text{low}$ .

Equivalently,  $i$  and  $i'$  overlap iff  $i.\text{low} \leq i'.\text{high}$  and  $i'.\text{low} \leq i.\text{high}$ . Memorize the positive form; it is two comparisons and it is what you write in code.

**The four steps applied:**

1. **Underlying structure:** a red-black tree, **keyed on**  $i.\text{low}$ .
2. **Additional information:**  $x.\text{max}$  = the maximum **high** endpoint anywhere in  $x$ 's subtree.
3. **Maintaining it:**  $x.\text{max} = \max(x.\text{int}.\text{high}, x.\text{left}.\text{max}, x.\text{right}.\text{max})$  — three values,  $O(1)$ , depends only on  $x$  and its children. **Theorem 17.1 applies**, so insert and delete stay  $O(\lg n)$ . (And Exercise 17.3-1 shows the update after a rotation is  $O(1)$ .) Sentinel:  $T.\text{nil}.\text{max} = -\infty$ .
4. **New operation:**

```
INTERVAL-SEARCH(T, i)
1  x = T.root
2  while x ≠ T.nil and i does not overlap x.int
3      if x.left ≠ T.nil and x.left.max ≥ i.low
4          x = x.left          // overlap in left subtree, or no
                                // overlap in right subtree
5      else x = x.right        // no overlap in left subtree
6  return x
```

→ C++ implementation: [A13 INTERVAL-SEARCH](#)

Each iteration is  $O(1)$  and the tree is  $O(\lg n)$  tall, so `INTERVAL-SEARCH` is  $O(\lg n)$ .

**Theorem 17.2.** Any execution of `INTERVAL-SEARCH(T, i)` either returns a node whose interval overlaps `i`, or returns `T.nil` and `T` contains no interval overlapping `i`.

*Proof skeleton.* The loop exits either on an overlap (obviously correct) or at `T.nil`. For the second case, show the search never takes a wrong turn:

**(1) If the search goes right** (`x.left == T.nil` or `x.left.max < i.low`), the **left subtree contains no overlap**. If it's empty, done. Otherwise, for any `i'` in `x`'s left subtree, `i'.high ≤ x.left.max < i.low`, so by the trichotomy `i'` lies entirely left of `i` — no overlap. ✓

**(2) If the search goes left** (`x.left.max ≥ i.low`), then *either* the left subtree contains an overlap (fine), *or* it doesn't — and then the **right subtree contains no overlap either**, so going left loses nothing. Proof of the second half: by definition of `max`, the left subtree contains some `i'` with `i'.high = x.left.max ≥ i.low`. By assumption `i'` does not overlap `i`, so by the trichotomy `i'.high < i'.low`. Now use that the tree is **keyed on low endpoints**: `i'` is in `x`'s left subtree so `i'.low ≤ x.int.low`, and any `i''` in `x`'s right subtree has `x.int.low ≤ i''.low`. Chaining:

```
i.high < i'.low ≤ x.int.low ≤ i''.low
```

so `i.high < i''.low` and by the trichotomy `i` and `i''` do not overlap. ✓ ■

**Notice what makes this work:** the augmentation (`max` of *high* endpoints) and the BST key (*low* endpoints) are **different fields**, and the proof needs both. That is the reason this proof is the one CLRS chooses to spell out — augmentation is not always as simple as "cache a subtree aggregate and read it off".

**Variants worth knowing:** Exercise 17.3-3 gets *all* `k` overlapping intervals in  $O(\min\{n, k \lg n\})$ ; the chapter notes cite a static structure achieving  $O(k + \lg n)$ . Exercise 17.3-6 uses a sweep line plus an interval tree to detect rectangle overlaps among `n` rectilinear rectangles in  $O(n \lg n)$  — a classic geometry-meets-search-tree problem (see M26).

## C++ Implementation — order-statistic tree and interval tree

Both are just an `Aug` plugged into the `RBTree` above.

```
// ---- order statistics -----  
-----
```

```

struct SizeAug {
    using Key = int;
    int size = 0; // sentinel keeps 0 (the
identity)
    template <class Node>
        void pull(const Node* node) { size = 1 + node->left->aug.size +
node->right->aug.size; }
};

using OrderStatisticTree = RBTree<SizeAug>;

template <class Tree>
typename Tree::Node* osSelect(const Tree& tree, int rank) { //
1-indexed
    typename Tree::Node* node = tree.root();
    while (node != tree.nil()) {
        int leftPlusSelf = node->left->aug.size + 1;
        if (rank == leftPlusSelf) return node;
        if (rank < leftPlusSelf) node = node->left;
        else { rank -= leftPlusSelf; node = node->right; }
    }
    return tree.nil();
}

template <class Tree>
int osRank(const Tree& tree, typename Tree::Node* node) {
    int leftPlusSelf = node->left->aug.size + 1;
    for (typename Tree::Node* climber = node; climber !=
tree.root(); climber = climber->parent)
        if (climber == climber->parent->right) leftPlusSelf +=
climber->parent->left->aug.size + 1;
    return leftPlusSelf;
}

// ---- intervals -----
-----
struct Interval {
    long long low = 0, high = 0;
};

inline bool operator<(const Interval& lhs, const Interval& rhs) {
return lhs.low < rhs.low; }

inline bool overlap(const Interval& lhs, const Interval& rhs) {

```

```

        return lhs.low <= rhs.high && rhs.low <= lhs.high;        //
interval trichotomy, positive form
}

struct IntervalAug {
    using Key = Interval;
    long long maxHigh = LLONG_MIN;        // sentinel keeps -
infinity (the identity)
    template <class Node>
    void pull(const Node* node) {
        maxHigh = max(node->key.high,
                        max(node->left->aug.maxHigh, node->right-
>aug.maxHigh));
    }
};

using IntervalTree = RBTree<IntervalAug>;

template <class Tree>
typename Tree::Node* intervalSearch(const Tree& tree, const
Interval& query) {
    typename Tree::Node* node = tree.root();
    while (node != tree.nil() && !overlap(node->key, query)) {
        if (node->left != tree.nil() && node->left->aug.maxHigh >=
query.low) node = node->left;
        else
            node = node->right;
    }
    return node;
}

```

*Verified:* order-statistic tree — for every  $i$  from 1 to  $n$ , `osSelect(i)` matched the  $i$ -th element of the reference sorted multiset, and `want[osRank(x) - 1] == x->key` for every node; subtree sizes re-derived bottom-up matched the stored `size` field at every checkpoint. Interval tree — over 4000 randomized insert/delete/query steps, `intervalSearch` agreed with brute-force scanning on *whether* an overlap exists, every returned node genuinely overlapped the query, and the stored `max` fields matched a bottom-up recomputation.

*Outside / Engineering Context — you rarely write these*

*In C++ the pragmatic answers are:*

- **Dictionary with order:** `std::map` / `std::set` (red-black in libstdc++ and libc++). `std::map::erase` invalidates only the erased iterator — the same "move the node, don't copy the key" guarantee as CLRS 4e. C++17 adds `extract()` / `merge()` for splicing nodes between containers without reallocating.
- **Order statistics:** GNU libstdc++ ships a policy-based tree with `find_by_order` / `order_of_key`:

```
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
using OST = tree<int, null_type, less<int>,
                rb_tree_tag,
                tree_order_statistics_node_update>;
```

*This is the competitive-programming shortcut. It is a GNU extension — not portable, not available in an interview's online judge unless you check.*

- **Offline order statistics on integer keys:** a **Fenwick tree over the value domain** (or over compressed coordinates) is simpler, faster, and cache-friendlier than any balanced tree. If the keys are bounded integers and you only need rank/select, reach for the BIT first.
- **Interval / range queries on a static set:** a **segment tree** or a sorted array with binary search beats an interval tree, which earns its keep only when the interval set is dynamic.

## Part 4 — B-Trees (CLRS 18)

### Why a different tree at all

### Unified Understanding

Every structure so far assumed the RAM model: any memory access costs  $O(1)$ . **B-trees exist because that assumption breaks at the disk boundary.**

A 7200 RPM magnetic disk takes 8.33 ms per rotation and about 4 ms average access time. Main memory is around 50 ns. That is **more than five orders of magnitude** — a computer could touch main memory over 100 000 times in the time it waits one rotation. So for disk-resident data we count **two** costs separately:

- the **number of disk accesses** (blocks read or written), and
- the **CPU time**.

Because a disk transfers whole **blocks** (typically 512–4096 bytes), and because latency is amortized over the whole block, you want each access to carry as much useful data as possible. Hence: **make a tree node exactly one disk block**.

A red-black tree node holds one key; a B-tree node holds hundreds or thousands. Both have height  $O(\lg n)$  in the abstract, but the *base of the logarithm* differs enormously. With branching factor 1001, a height-2 B-tree stores over a **billion** keys — 1 node at depth 0, 1001 at depth 1, 1 002 001 leaves at depth 2, 1000 keys each. Keep the root pinned in memory and **at most two disk accesses** find any key in that billion-key tree. B-trees save a factor of about  $\lg t$  over red-black trees in nodes examined.

The pseudocode assumes an explicit `DISK-READ(x)` before touching `x`'s attributes and `DISK-WRITE(x)` after modifying them, with the pattern:

```
x = a pointer to some object
DISK-READ(x)
operations that access and/or modify x's attributes
DISK-WRITE(x)          // omitted if nothing was modified
other operations that only read x
```

Two standing conventions in the chapter: **the root is always in main memory** (never needs a `DISK-READ`, but must be `DISK-WRITE` if changed), and **any node passed as a parameter has already been read in**. All the procedures are **one-pass, top-down** — they never back up.

***Note the reframing.** This is the same lesson as M07's §11.5 (memory hierarchy and linear probing) and M05's external sorting war story: the cost model determines the data structure. Change  $O(1)$  memory access to "a block costs 4 ms", and the optimal dictionary changes shape completely.*

## Definition

A B-tree `T` is a rooted tree with root `T.root` such that:

1. Every node `x` has: `x.n` (number of keys currently stored), the keys `x.key1 ≤ x.key2 ≤ ... ≤ x.key_{x.n}` in monotonically increasing order, and a boolean `x.leaf`.

2. Each **internal** node  $x$  also has  $x.n + 1$  child pointers  $x.c_1, \dots, x.c_{\{x.n+1\}}$ .  
Leaves have no children.
3. The keys **separate** the child ranges: if  $k_i$  is any key in the subtree rooted at  $x.c_i$ , then

$$k_1 \leq x.key_1 \leq k_2 \leq x.key_2 \leq \dots \leq x.key_{\{x.n\}} \leq k_{\{x.n+1\}}$$

4. **All leaves have the same depth**, which is the tree's height  $h$ .

5. There is a fixed integer  $t \geq 2$ , the **minimum degree**, bounding node occupancy:

- every node **other than the root** has at least  $t - 1$  keys, so every internal non-root node has at least  $t$  children; a nonempty root has at least 1 key;
- every node has **at most  $2t - 1$  keys**, so at most  $2t$  children. A node with exactly  $2t - 1$  keys is **full**.

$t = 2$  gives the **2-3-4 tree**: every internal node has 2, 3, or 4 children. In practice  $t$  is chosen so a node fills a disk block, giving branching factors of **50 to 2000**.

**Variants:** a **B<sup>+</sup>-tree** stores all satellite data in the leaves and only keys + child pointers internally, maximizing internal branching factor — this is what real databases use. A **B\*-tree** requires internal nodes to be  $\frac{2}{3}$  full rather than  $\frac{1}{2}$  full.

## Theorem 18.1 — height

**Theorem 18.1.** If  $n \geq 1$ , then for any  $n$ -key B-tree of height  $h$  and minimum degree  $t \geq 2$ :

$$h \leq \log_t ((n + 1) / 2)$$

*Proof skeleton.* Minimize the node count at each depth. The root has  $\geq 1$  key so  $\geq 2$  children; every other internal node has  $\geq t$  children. So there are  $\geq 2$  nodes at depth 1,  $\geq 2t$  at depth 2,  $\geq 2t^2$  at depth 3, ...,  $\geq 2t^{h-1}$  at depth  $h$ . Each non-root node holds  $\geq t - 1$  keys:

$$\begin{aligned} n &\geq 1 + (t - 1) \cdot \sum_{i=1..h} 2t^{i-1} \\ &= 1 + 2(t - 1) \cdot (t^h - 1)/(t - 1) && \text{[geometric sum]} \\ &= 2t^h - 1 \end{aligned}$$



So  $t^h \leq (n + 1)/2$ ; take base- $t$  logs. ■

Same  $O(\lg n)$  shape as a red-black tree — but with base  $t$  instead of base 2, which is the entire point.

*Verified:* a B-tree with  $t = 3$  holding 1032 keys after 60 000 random operations had height 4, against the bound  $\log_3(1033/2) = 5.69$ ; all leaves were at equal depth and every non-root node held between 2 and 5 keys.

## Searching

```
B-TREE-SEARCH(x, k)
1  i = 1
2  while i ≤ x.n and k > x.keyi
3      i = i + 1
4  if i ≤ x.n and k == x.keyi
5      return (x, i)
6  elseif x.leaf
7      return NIL
8  else DISK-READ(x.ci)
9      return B-TREE-SEARCH(x.ci, k)
```

→ **C++ implementation:** [A14 B-TREE-SEARCH](#)

An  $(x.n + 1)$ -way branch instead of a 2-way branch. Lines 1–3 find the smallest  $i$  with  $k \leq x.key_i$ , or set  $i = x.n + 1$ .

**Complexity:**  $O(h) = O(\log_t n)$  **disk accesses.** CPU time is  $O(t)$  per node (linear scan) for  $O(t \cdot h) = O(t \log_t n)$  total. Exercise 18.2-6: replacing the linear scan with a binary search inside the node makes the CPU time  $O(\lg n)$  **independent of  $t$**  — which is what real implementations do, since  $t$  can be in the thousands.

## Insertion — split full nodes on the way down

You cannot insert into a full leaf, and splitting a full leaf may overflow its parent, which may overflow *its* parent — a cascade back up the tree. The B-tree trick removes the cascade entirely:

*Split every full node you encounter as you descend. Then whenever you need to split a node, its parent is guaranteed non-full, so the split's median key always has room to move up. One downward pass, no backtracking.*

**Splitting** takes a full child  $y$  with  $2t - 1$  keys and cuts it into two nodes of  $t - 1$  keys each, with the median key  $y.key_t$  moving **up into the parent** as the separator.

```

B-TREE-SPLIT-CHILD(x, i)          // x nonfull, x.ci full; both in
memory
1  y = x.ci
2  z = ALLOCATE-NODE()
3  z.leaf = y.leaf
4  z.n = t - 1
5  for j = 1 to t - 1             // z takes y's greatest t-1 keys
6      z.keyj = y.key{j+t}
7  if not y.leaf
8      for j = 1 to t             // ...and the corresponding t
children
9      z.cj = y.c{j+t}
10 y.n = t - 1                   // y keeps the smallest t-1 keys
11 for j = x.n + 1 downto i + 1 // shift x's children right
12     x.c{j+1} = x.cj
13 x.c{i+1} = z
14 for j = x.n downto i          // shift x's keys right
15     x.key{j+1} = x.keyj
16 x.keyi = y.keyt              // the median moves up
17 x.n = x.n + 1
18 DISK-WRITE(y) ; DISK-WRITE(z) ; DISK-WRITE(x)

```

→ **C++ implementation:** [A15 B-TREE-SPLIT-CHILD](#)

$\Theta(t)$  CPU time,  $O(1)$  disk operations.

|                                                                                                                                                                           |                                                                                                                                                                         |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> B-TREE-INSERT(T, k) 1  r = T.root 2  if r.n == 2t - 1 3      s = B-TREE-SPLIT-ROOT(T) 4      B-TREE-INSERT-NONFULL(s, k) 5  else B-TREE-INSERT-NONFULL(r, k) </pre> | <pre> B-TREE-SPLIT-ROOT(T) 1  s = ALLOCATE-NODE() 2  s.leaf = FALSE 3  s.n = 0 4  s.c<sub>1</sub> = T.root 5  T.root = s 6  B-TREE-SPLIT-CHILD(s, 1) 7  return s </pre> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

→ C++ implementation: [A16 B-TREE-INSERT and B-TREE-SPLIT-ROOT](#)

```
B-TREE-INSERT-NONFULL(x, k)
1  i = x.n
2  if x.leaf
3      while i ≥ 1 and k < x.keyi      // shift to make room
4          x.key_{i+1} = x.keyi ; i = i - 1
6      x.key_{i+1} = k ; x.n = x.n + 1
8      DISK-WRITE(x)
9  else while i ≥ 1 and k < x.keyi      // find the child
10     i = i - 1
11     i = i + 1
12     DISK-READ(x.ci)
13     if x.ci.n == 2t - 1                // split it BEFORE
descending
14         B-TREE-SPLIT-CHILD(x, i)
15         if k > x.keyi i = i + 1      // which half does k belong
in?
17     B-TREE-INSERT-NONFULL(x.ci, k)
```

→ C++ implementation: [A17 B-TREE-INSERT-NONFULL](#)

**Splitting the root is the only way a B-tree grows taller** — and unlike a BST, a **B-tree grows at the top, not at the bottom**. That is why all leaves stay at the same depth: every leaf gains a level at the same instant.

**Complexity:**  $O(h)$  disk accesses ( $O(1)$  reads/writes per level),  $O(t \cdot h) = O(t \log_t n)$  CPU. `B-TREE-INSERT-NONFULL` is tail-recursive, so it converts to a `while` loop, proving only  $O(1)$  blocks need to be resident at any time.

## Deletion — fill thin nodes on the way down

The mirror image of insertion. Insertion prevents nodes from becoming **overfull** while descending; deletion prevents them from becoming **underfull** while descending.

**The invariant that makes it one pass:** whenever `B-TREE-DELETE` recurses into a node `x`, `x` has **at least  $t$  keys** — one *more* than the B-tree minimum of  $t - 1$ . That slack means a key can always be pushed down out of `x` without violating the minimum. (The root is exempt and is never the target of a recursive call.)

Because of that invariant, **B-TREE-DELETE** combines the search with the deletion rather than being handed a node — unlike **TREE-DELETE** and **RB-DELETE**.

Three cases, by what the search finds at node  $x$ :

**Case 1 —  $x$  is a leaf.** If  $k \in x$ , remove it. If not,  $k$  was never in the tree; done.

**Case 2 —  $x$  is internal and  $k = x.key_i$ .** Let  $y = x.c_i$  (child before  $k$ ) and  $z = x.c_{i+1}$  (child after  $k$ ).

- **2a.**  $y$  has  $\geq t$  keys. Find  $k' = \text{predecessor}$  of  $k$  in  $y$ 's subtree, replace  $k$  with  $k'$  in  $x$ , and recursively delete  $k'$  from  $y$ .
- **2b.**  $y$  has  $t - 1$  keys but  $z$  has  $\geq t$ . Symmetric: use the **successor**.
- **2c.** Both  $y$  and  $z$  have  $t - 1$  keys. **Merge**  $k$  and all of  $z$  into  $y$  (giving  $y$  exactly  $2t - 1$  keys), free  $z$ , and recursively delete  $k$  from  $y$ .

**Case 3 —  $x$  is internal and  $k \notin x$ .** Determine the child  $x.c_i$  whose subtree must contain  $k$ . If  $x.c_i$  has only  $t - 1$  keys, fix that *before* descending:

- **3a.**  $x.c_i$  has an immediate sibling with  $\geq t$  keys. **Rotate**: move a key from  $x$  down into  $x.c_i$ , move a key from the sibling up into  $x$ , and move the appropriate child pointer from the sibling into  $x.c_i$ .
- **3b.**  $x.c_i$  and *all* its immediate siblings have  $t - 1$  keys. **Merge**  $x.c_i$  with one sibling, pulling a key down from  $x$  to be the median of the merged node.

Then recurse on the appropriate child.

**Height shrinks at the top too.** In cases 2c and 3b, if  $x$  is the root it can end up with zero keys; then delete the root and make its only child  $x.c_1$  the new root. That is the *only* way a B-tree gets shorter — mirroring the fact that root splitting is the only way it gets taller.

**Complexity:**  $O(h)$  disk operations,  $O(t \cdot h) = O(t \log_t n)$  CPU. Cases 2a and 2b look like they need a return trip up, but the code just keeps a pointer to  $x$  and the key index and writes the predecessor/successor directly into place — no re-traversal.

## C++ Implementation — in-memory B-tree

Here  $t$  is a compile-time template parameter so the key/child arrays are fixed-size `std::array` — the layout a real implementation would map onto a disk block.

```

#include <array>
#include <vector>

template <class Key, int MinDegree = 3>
class BTree {
    static_assert(MinDegree >= 2, "minimum degree must be at least
2");
public:
    struct Node {
        int keyCount = 0;
        bool leaf = true;
        array<Key, 2 * MinDegree - 1> keys{};
        array<Node*, 2 * MinDegree> children{};
    };

    BTree() { root_ = new Node(); }
    ~BTree() { destroy(root_); }
    BTree(const BTree&) = delete;
    BTree& operator=(const BTree&) = delete;

    Node* root() const { return root_; }

    bool contains(const Key& key) const { return search(root_, key)
!= nullptr; }

    void insert(const Key& key) {
        if (root_>keyCount == 2 * MinDegree - 1) splitRoot();
// grow at the top
        insertNonfull(root_, key);
    }

    void erase(const Key& key) {
        eraseFrom(root_, key);
        if (root_>keyCount == 0 && !root_>leaf) { //
shrink at the top
            Node* oldRoot = root_;
            root_ = root_>children[0];
            delete oldRoot;
        }
    }

    void inorder(vector<Key>& out) const { walk(root_, out); }

```

```

private:
    Node* root_;

    static const Node* search(const Node* node, const Key& key) {
        int i = 0;
        while (i < node->keyCount && node->keys[i] < key) ++i;
        if (i < node->keyCount && !(key < node->keys[i])) return
node;
        return node->leaf ? nullptr : search(node->children[i],
key);
    }

    void splitRoot() {
        Node* newRoot = new Node();
        newRoot->leaf = false;
        newRoot->children[0] = root_;
        root_ = newRoot;
        splitChild(newRoot, 0);
    }

    // parent is nonfull, parent->children[childIndex] is full:
    // split that child around its median key
    static void splitChild(Node* parent, int childIndex) {
        Node* fullChild = parent->children[childIndex];
        Node* newSibling = new Node();
        newSibling->leaf = fullChild->leaf;
        newSibling->keyCount = MinDegree - 1;
        for (int j = 0; j < MinDegree - 1; ++j)
            newSibling->keys[j] = fullChild->keys[j + MinDegree];
        if (!fullChild->leaf)
            for (int j = 0; j < MinDegree; ++j)
                newSibling->children[j] = fullChild->children[j +
MinDegree];

        const Key median = fullChild->keys[MinDegree - 1];
        fullChild->keyCount = MinDegree - 1;
        for (int j = parent->keyCount; j >= childIndex + 1; --j)
            parent->children[j + 1] = parent->children[j];
        parent->children[childIndex + 1] = newSibling;
        for (int j = parent->keyCount - 1; j >= childIndex; --j)
            parent->keys[j + 1] = parent->keys[j];
        parent->keys[childIndex] = median;
    }

```

```

        ++parent->keyCount;
    }

    static void insertNonfull(Node* node, const Key& key) {
        int i = node->keyCount - 1;
        if (node->leaf) {
            while (i >= 0 && key < node->keys[i]) { node->keys[i +
1] = node->keys[i]; --i; }
            node->keys[i + 1] = key;
            ++node->keyCount;
            return;
        }
        while (i >= 0 && key < node->keys[i]) --i;
        ++i;
        if (node->children[i]->keyCount == 2 * MinDegree - 1) {
            // split BEFORE descending
            splitChild(node, i);
            if (node->keys[i] < key) ++i;
        }
        insertNonfull(node->children[i], key);
    }

    static Key maxKey(Node* node) { while (!node->leaf) node =
node->children[node->keyCount]; return node->keys[node->keyCount -
1]; }

    static Key minKey(Node* node) { while (!node->leaf) node =
node->children[0]; return node->keys[0]; }

    // merge children[i], keys[i] and children[i+1] into
children[i]
    static void mergeChildren(Node* node, int i) {
        Node* leftChild = node->children[i];
        Node* rightChild = node->children[i + 1];
        leftChild->keys[MinDegree - 1] = node->keys[i];
        for (int j = 0; j < rightChild->keyCount; ++j)
            leftChild->keys[j + MinDegree] = rightChild->keys[j];
        if (!leftChild->leaf)
            for (int j = 0; j <= rightChild->keyCount; ++j)
                leftChild->children[j + MinDegree] = rightChild-
>children[j];
        leftChild->keyCount = 2 * MinDegree - 1;
    }

```

```

        for (int j = i; j + 1 < node->keyCount; ++j) node->keys[j]
= node->keys[j + 1];
        for (int j = i + 1; j < node->keyCount; ++j) node-
>children[j] = node->children[j + 1];
        --node->keyCount;
        delete rightChild;
    }

    static void borrowFromLeft(Node* node, int i) {          // case
3a

        Node* child = node->children[i];
        Node* leftSibling = node->children[i - 1];
        for (int j = child->keyCount - 1; j >= 0; --j) child-
>keys[j + 1] = child->keys[j];
        if (!child->leaf)
            for (int j = child->keyCount; j >= 0; --j) child-
>children[j + 1] = child->children[j];
        child->keys[0] = node->keys[i - 1];
        if (!child->leaf) child->children[0] = leftSibling-
>children[leftSibling->keyCount];
        node->keys[i - 1] = leftSibling->keys[leftSibling->keyCount
- 1];
        ++child->keyCount;
        --leftSibling->keyCount;
    }

    static void borrowFromRight(Node* node, int i) {          // case
3a, mirrored

        Node* child = node->children[i];
        Node* rightSibling = node->children[i + 1];
        child->keys[child->keyCount] = node->keys[i];
        if (!child->leaf) child->children[child->keyCount + 1] =
rightSibling->children[0];
        node->keys[i] = rightSibling->keys[0];
        for (int j = 0; j + 1 < rightSibling->keyCount; ++j)
rightSibling->keys[j] = rightSibling->keys[j + 1];
        if (!rightSibling->leaf)
            for (int j = 0; j < rightSibling->keyCount; ++j)
rightSibling->children[j] = rightSibling->children[j + 1];
        ++child->keyCount;
        --rightSibling->keyCount;
    }

```



```

    // precondition: node is the root, or node->keyCount >=
MinDegree
    static void eraseFrom(Node* node, const Key& key) {
        int i = 0;
        while (i < node->keyCount && node->keys[i] < key) ++i;
        const bool foundHere = (i < node->keyCount && !(key < node-
>keys[i]));

        if (node->leaf) { //
case 1
            if (!foundHere) return; //
key not in the tree
            for (int j = i; j + 1 < node->keyCount; ++j) node-
>keys[j] = node->keys[j + 1];
            --node->keyCount;
            return;
        }
        if (foundHere) { // case
2
            Node* leftChild = node->children[i];
            Node* rightChild = node->children[i + 1];
            if (leftChild->keyCount >= MinDegree) { // 2a:
predecessor
                const Key pred = maxKey(leftChild);
                node->keys[i] = pred;
                eraseFrom(leftChild, pred);
            } else if (rightChild->keyCount >= MinDegree) { // 2b:
successor
                const Key succ = minKey(rightChild);
                node->keys[i] = succ;
                eraseFrom(rightChild, succ);
            } else { // 2c:
merge, then recurse
                mergeChildren(node, i);
                eraseFrom(leftChild, key);
            }
            return;
        }
        // case 3: descend, but only into a child with at least
MinDegree keys
        if (node->children[i]->keyCount == MinDegree - 1) {

```

```

        if (i > 0 && node->children[i - 1]->keyCount >=
MinDegree)
            borrowFromLeft(node, i);    // 3a
        else if (i < node->keyCount && node->children[i + 1]-
>keyCount >= MinDegree) borrowFromRight(node, i);    // 3a
        else if (i < node->keyCount)
            mergeChildren(node, i);    // 3b
        else
        {
            mergeChildren(node, i - 1); --i; }
        }
        eraseFrom(node->children[i], key);
    }

    static void walk(const Node* node, vector<Key>& out) {
        for (int i = 0; i < node->keyCount; ++i) {
            if (!node->leaf) walk(node->children[i], out);
            out.push_back(node->keys[i]);
        }
        if (!node->leaf) walk(node->children[node->keyCount], out);
    }

    static void destroy(Node* node) {
        if (!node) return;
        if (!node->leaf) for (int i = 0; i <= node->keyCount; ++i)
destroy(node->children[i]);
        delete node;
    }
};

```

### Implementation notes.

- All comparisons go through `operator<` only (`!(key < node->keys[i])` for  $\geq$ ), which is the C++ convention — one comparator, no `operator==` requirement.
- `eraseFrom`'s precondition ("`node` is the root, or `node->keyCount  $\geq$  MinDegree`") is the CLRS invariant made explicit as a comment. Every recursive call site establishes it: 2a/2b descend into a child with  $\geq t$  keys, 2c descends into a freshly merged node with  $2t - 1$  keys, case 3 fixes the child *before* descending.
- The `else { mergeChildren(node, i - 1); --i; }` branch is the "no right sibling" corner (`i == node->keyCount`): merge with the **left** sibling and shift the target index down. Getting this one wrong is the single most common B-tree deletion bug.

Verified: 60 000 randomized insert/erase operations against `std::set` with `t = 3` — `contains` agreed on every step; at every checkpoint every non-root node held 2–5 keys, keys within each node were strictly increasing, and all leaves were at equal depth; the final inorder walk matched `std::set` exactly.

## Outside / Engineering Context

- ***B<sup>+</sup>-trees are the index structure of essentially every relational database and filesystem*** (InnoDB, PostgreSQL btree indexes, NTFS, ext4's HTree, APFS). Satellite data in leaves, leaves chained in a linked list so a range scan is sequential.
- ***The modern rival is the LSM-tree*** (LevelDB, RocksDB, Cassandra): buffer writes in memory, flush sorted runs, compact in the background. LSMs win on write throughput and space amplification; B-trees win on read latency and range scans. Choosing between them is a real system-design question.
- ***Cache-oblivious B-trees*** (Bender, Demaine, Farach-Colton, cited in the chapter notes) get good memory-hierarchy behavior without knowing the block size — relevant because a modern machine has 4–5 levels of hierarchy, not one.
- ***In-memory, `t` is chosen for the cache line, not the disk block.*** A B-tree with nodes sized to a few cache lines often beats `std::map` for lookup-heavy in-memory workloads, purely because of locality — the same reason linear probing beat chaining in M07.

---

## Recognition Patterns

| SIGNAL IN THE PROBLEM STATEMENT                                         | STRUCTURE                                                                                                               |
|-------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| "insert, delete, look up" and nothing about order                       | <b>Hash table</b> (M07) — a tree is strictly worse here                                                                 |
| "k-th smallest / largest", "median of a stream", "rank of x"            | <b>Order-statistic tree</b> , or a Fenwick tree over compressed values                                                  |
| "next larger element after x", "floor/ceiling", "predecessor/successor" | <b>Balanced BST</b> ( <code>std::map::lower_bound</code> )                                                              |
| "all elements between a and b"                                          | Balanced BST range walk, $O(k + \lg n)$                                                                                 |
| "which meetings conflict", "does any segment overlap", "stabbing query" | <b>Interval tree</b> (or a sweep line if the set is static)                                                             |
| "keep a sorted collection under insertions and deletions"               | Balanced BST — this is the definitional case                                                                            |
| "top k so far" with only insert and extract-max                         | <b>Heap</b> (M05), not a tree — cheaper and simpler                                                                     |
| Data doesn't fit in memory / is on disk / is a database index           | <b>B-tree / B<sup>+</sup>-tree</b>                                                                                      |
| Keys are bounded small integers and you need rank                       | <b>Fenwick / segment tree</b> , not a BST                                                                               |
| Access pattern is highly skewed (a few hot keys)                        | <b>Splay tree</b> — amortized $O(\lg n)$ with excellent constants on skewed workloads                                   |
| You need to keep old versions of the set around                         | <b>Persistent BST</b> (CLRS Problem 13-1): copy only the root-to-node path, $O(\lg n)$ time <i>and space</i> per update |
| "merge two sets where all of one is less than all of the other"         | <b>Join</b> (Problem 13-2): $O(\lg n)$ on red-black trees using stored black-heights                                    |

**The counter-recognition that matters most in interviews:** if the problem needs *only* membership and no ordering, and someone reaches for a tree, that is a  $\lg n$  factor thrown away. And conversely, if a problem needs successor/rank/range and someone reaches for a hash table, that is  $\Theta(n)$  per query. **Ordering is the discriminator.**

## Interview Checklist

Before you write a line of tree code:

1. **Do I actually need order?** If not, hash table.
2. **Is the key set static?** If so, a sorted array + binary search beats every tree — no pointers, perfect locality.
3. **Can I use the library?** `std::map`/`std::set` is the right answer in 95% of interviews. Say so, then implement only what the interviewer asks for.
4. **Do I need balance?** If insertion order is adversarial or sorted, yes. Say the words: *"a plain BST degrades to  $O(n)$  on sorted input; I'd use `std::map`, or a treap if I have to hand-roll it."*
5. **Is there an augmentation?** Subtree size for rank/select, subtree max for intervals, subtree sum for range sums. Cite Theorem 17.1's condition: *the field must be computable in  $O(1)$  from the node and its two children.*
6. **Am I about to write a red-black tree by hand?** Almost never the right call under time pressure. A **treap** or **randomized BST** gets the same expected bounds in ~40 lines with no cases; say why you're choosing it.

**Things to be able to say in one sentence each:**

- Why the new node is colored red on insertion.
  - Why insert does  $\leq 2$  rotations and delete  $\leq 3$ , and why that matters for augmentation.
  - What "doubly black" means and where the extra black lives.
  - Why B-trees split on the way down instead of on the way up.
  - Why B-trees grow and shrink at the root, and why that keeps all leaves at equal depth.
  - Why the interval tree keys on `low` but augments with `max` of `high`.
- 

## Complexity Summary

| OPERATION                  | PLAIN BST                                                           | RED-BLACK TREE                                | B-TREE (MIN. DEGREE $t$ )                                                              |
|----------------------------|---------------------------------------------------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------|
| SEARCH                     | $O(h)$ , $h \in [\lg n, n]$                                         | $O(\lg n)$                                    | $O(\log_t n)$ disk, $O(t \log_t n)$ CPU (or $O(\lg n)$ CPU with in-node binary search) |
| MIN / MAX / SUCC / PRED    | $O(h)$                                                              | $O(\lg n)$                                    | $O(\log_t n)$                                                                          |
| INSERT                     | $O(h)$                                                              | $O(\lg n)$ , $\leq 2$ rotations               | $O(\log_t n)$ disk                                                                     |
| DELETE                     | $O(h)$                                                              | $O(\lg n)$ , $\leq 3$ rotations               | $O(\log_t n)$ disk                                                                     |
| Inorder walk               | $\Theta(n)$                                                         | $\Theta(n)$                                   | $\Theta(n)$                                                                            |
| Height                     | $\lceil \lg(n+1) \rceil \dots n$ ; $\Theta(\lg n)$ w.h.p. if random | $\leq 2 \lg(n+1)$                             | $\leq \log_t((n+1)/2)$                                                                 |
| Space                      | $\Theta(n)$ , 3 pointers/node                                       | $\Theta(n)$ , 3 pointers + 1 bit              | $\Theta(n)$ , but nodes are block-sized                                                |
| Extra ops via augmentation | —                                                                   | SELECT / RANK / interval search in $O(\lg n)$ | same idea, per-node aggregates                                                         |

Recursion-stack space:  $O(h)$  for the recursive walks — worth converting the hot ones (search, min, insert descent) to iterative loops, which is what every implementation above does.

## One-Page Recall

- **BST property:** left  $\leq$  node  $\leq$  right. Everything costs  $O(h)$ . Inorder walk = sorted,  $\Theta(n)$ .
- **Deletion has three shapes:** 0 children (unlink), 1 child (splice), 2 children (replace with successor, then delete the successor which has  $\leq 1$  child). **TRANSPLANT** is the helper that makes it readable. CLRS 4e **moves the node**, doesn't copy the key, so external handles stay meaningful.
- **Height is set by insertion order, which you don't control.** Sorted input  $\rightarrow$  height  $n$ . Random order  $\rightarrow \Theta(\lg n)$  w.h.p.
- **Red-black:** 5 properties. Root black, leaves (sentinel) black, red node  $\Rightarrow$  black children, equal black-height on all downward paths.  $\Rightarrow h \leq 2 \lg(n+1)$ .

- **Rotation** =  $O(1)$  shape change that preserves inorder order. It is the only structural tool.
  - **Insert fixup**: color new node red; 3 cases keyed on the **uncle**. Uncle red → recolor and climb 2 levels (only looping case). Uncle black → 1–2 rotations and stop.  $\leq 2$  rotations.
  - **Delete fixup**: the removed/moved node's blackness becomes an **extra black on a pointer**; 4 cases keyed on the **sibling** and its children. Sibling black with two black children → push the extra black up (only looping case).  $\leq 3$  rotations.
  - **Augmentation (4 steps)**: pick structure → pick field → verify maintenance → build the new operation. **Theorem 17.1**: if  $x.f$  is computable in  $O(1)$  from  $x$ ,  $x.left$ ,  $x.right$ , maintenance is free — *because RB trees do  $O(1)$  rotations per operation*.
  - **Order-statistic tree**:  $x.size = x.left.size + x.right.size + 1$ ,  $T.nil.size = 0$ . OS-SELECT descends subtracting  $r$  when going right; OS-RANK ascends adding  $y.p.left.size + 1$  when  $y$  is a right child.
  - **Interval tree**: keyed on  $low$ , augmented with  $max$  of  $high$ . Overlap  $\iff a.low \leq b.high \ \&\& \ b.low \leq a.high$ . Search goes left iff  $x.left.max \geq i.low$ ; correctness needs both the key and the augmentation (Theorem 17.2).
  - **B-tree**: minimum degree  $t$ ; every non-root node has  $t-1$  to  $2t-1$  keys; all leaves at equal depth;  $h \leq \log_t((n+1)/2)$ . Insert splits full nodes on the way down; delete fills thin nodes on the way down; both are single passes. The tree grows and shrinks **at the root**. Branching factor 50–2000 in practice because a node is one disk block.
  - **The meta-lesson**: the cost model chooses the structure. RAM model → red-black. Block-transfer model → B-tree. Skewed accesses → splay. Only membership → hash table.
- 

## Practice — where to drill this module

| IDEA IN THIS MODULE                        | PROBLEM                                                         | WHY IT'S THE RIGHT DRILL                                                                                                           |
|--------------------------------------------|-----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| The BST property, precisely                | <a href="#">98 · Validate Binary Search Tree</a>                | the naive "check parent vs children" answer is <b>wrong</b> ; you need a range, which is the property stated correctly             |
| <code>INORDER-TREE-WALK</code>             | <a href="#">94 · Binary Tree Inorder Traversal</a>              | recursive, then iterative with an explicit stack, then Morris — three ways to say the same thing                                   |
| <code>TREE-SUCCESSOR</code> as an iterator | <a href="#">173 · Binary Search Tree Iterator</a>               | <code>next()</code> in amortized $O(1)$ is <code>TREE-SUCCESSOR</code> ; this is <a href="#">A3</a> as a submission                |
| <code>TREE-DELETE</code> , all four cases  | <a href="#">450 · Delete Node in a BST</a>                      | the two-children case is where everyone gets it wrong                                                                              |
| Order statistics                           | <a href="#">230 · Kth Smallest Element in a BST</a>             | the "what if it's called often?" follow-up is asking for the <code>size</code> augmentation — <a href="#">A11</a>                  |
| Augmentation, live                         | <a href="#">1649 · Create Sorted Array through Instructions</a> | rank queries under insertion; a BIT is the easy answer, an OS-tree the general one                                                 |
| Interval overlap                           | <a href="#">729 · My Calendar I</a>                             | <code>INTERVAL-SEARCH</code> with a <code>std::set</code> and <code>lower_bound</code> — the practical form of <a href="#">A13</a> |
| Interval overlap, harder                   | <a href="#">715 · Range Module</a>                              | dynamic intervals with merging; where an interval tree earns its keep                                                              |
| Balanced-tree behaviour                    | <a href="#">1206 · Design Skiplist</a>                          | a randomized alternative to red-black; same $O(\lg n)$ , far less code                                                             |

**Beyond LeetCode.** [CSES Problem Set](#) — *Range Queries* and *Tree Algorithms*. [Codeforces trees tag](#) · [data structures tag](#).

**The honest advice for interviews:** you will almost never be asked to write `RB-INSERT-FIXUP`. You *will* be asked what `std::map` guarantees, why it is  $O(\lg n)$ , when to prefer `unordered_map`, and how you would add order statistics to a balanced tree. Learn [A1](#) – [A5](#) and [A11](#) – [A13](#) cold; read [A6](#) – [A10](#) twice and know the *shape* of the argument.

## C++ Toolkit for This Module

Language material from Weiss, *Data Structures and Algorithm Analysis in C++*, 4th ed., ch. 4 and §1.5–1.6.



## 1. Interface and implementation, separated

Weiss [§1.4.3, p.16] presents every class as an interface followed by out-of-line member definitions. This appendix follows that convention exactly: the class declarations come first, then each pseudocode block's implementation is given as an out-of-line definition:

```
struct Example {  
    int f(int x);           // declaration, in the class  
};  
int Example::f(int x) {     // definition, outside -- note the  
    Example::               // Example:: qualifier  
    return x + 1;  
}
```

That is what makes it possible for each appendix entry below to be a self-contained code block that still compiles as part of one program.

## 2. The sentinel `nil` node, and why red-black trees need it

A1–A5 (plain BST) use `nullptr` for "no child". A6–A13 (red-black) use a shared **sentinel** `nil_` node instead, for two reasons that are specific to red-black trees:

- **`nil` must have a colour.** `RB-DELETE-FIXUP` reads `w.left.color` and `w.right.color`, where those children may be absent. With `nullptr` that is a crash; with a sentinel coloured `BLACK` it is exactly the right answer.
- **`nil` must have a parent.** `RB-TRANSPLANT` sets `v.p = u.p` *unconditionally*, even when `v` is `nil`, because `RB-DELETE-FIXUP` then walks up from `x` — and `x` can be `nil`.

One sentinel is shared by every leaf position in the tree, so the cost is a single node (M06's sentinel discussion, applied where it genuinely pays).

## 3. `enum class` for colours

```
enum class Color { Red, Black };           // scoped: Color::Red, and no  
implicit int conversion
```

A plain `enum { RED, BLACK }` leaks its enumerators into the enclosing scope and converts silently to `int`, so `if (x->color)` compiles and means something accidental. `enum class` makes both mistakes compile errors.

## 4. Raw owning pointers, and the Big-Five — again

Every structure here allocates nodes with `new`. Weiss's rule from M06 applies unchanged: **a class holding raw owning pointers must declare all five special members**, because the compiler's shallow copy would give two trees sharing nodes and then a double free. The classes below `= delete` their copy operations for that reason. A production tree would implement a deep copy; a notes implementation is better off making the mistake impossible.

## 5. struct vs class

`struct` members default to `public`, `class` members to `private`. Nothing else differs. The convention used below: `struct` for the plain node types (they are transparent data), `class` for the trees (they maintain invariants and must control access).

## 6. Recursion depth is a real constraint

`INORDER-TREE-WALK` recurses to depth `h`. For a **balanced** tree  $h = O(\lg n)$  and nothing can go wrong. For an unbalanced BST built from sorted input,  $h = n$  — and at  $n \approx 10^5$  that overflows the default 8 MB stack and **segfaults**. This is not a theoretical worry: "insert 1..n in order" is the most natural test input anyone writes.

## 7. What the standard library already gives you

| CLRS STRUCTURE       | C++                                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------------------|
| red-black tree       | <code>std::map</code> , <code>std::set</code> , <code>std::multimap</code> , <code>std::multiset</code> |
| B-tree               | <i>(none)</i> — this is a disk structure; databases implement it                                        |
| order-statistic tree | <i>(none portable)</i> — GNU <code>__gnu_pbds::tree</code> on libstdc++ only                            |
| interval tree        | <i>(none)</i> — <code>std::map</code> + <code>lower_bound</code> covers the common cases                |

`std::map::erase` moves nodes rather than copying keys, which is exactly the CLRS 4e `TREE-DELETE` guarantee discussed in A5. C++17 adds `extract()` and `merge()`, which splice nodes between containers with no allocation — the same "move the node, not the key" idea made public.

---

## Appendix — C++ for Every Pseudocode Block

**Structure declarations.** Every entry below is an out-of-line definition of one of these members (toolkit §1).

```
// ----- plain BST (A1-A5) -----
struct BstNode {
    int key;
    BstNode* left  = nullptr;
    BstNode* right = nullptr;
    BstNode* parent = nullptr;          // TREE-SUCCESSOR needs to
    climb upward
    explicit BstNode(int key) : key(key) {}
};

class Bst {
public:
    Bst() = default;
    ~Bst() { destroy(root_); }
    Bst(const Bst&) = delete;          // owns raw pointers
    (toolkit 4)
    Bst& operator=(const Bst&) = delete;

    void      inorderWalk(BstNode* node, vector<int>& out) const;
    // A1
    BstNode* search(BstNode* node, int key) const;
    // A2
    BstNode* iterativeSearch(BstNode* node, int key) const;
    // A2
    BstNode* minimum(BstNode* node) const;
    // A2
    BstNode* maximum(BstNode* node) const;
    // A2
    BstNode* successor(BstNode* node) const;
    // A3
    void      transplant(BstNode* oldSubtree, BstNode* newSubtree);
    // A4
    void      erase(BstNode* target);
    // A5
```

```

    // An ordinary BST insert: descend to a leaf position, then
link. Included
    // so the appendix is runnable; CLRS gives it as TREE-INSERT.
    BstNode* insert(int key) {
        BstNode* parent = nullptr;           // trails one step
behind the walker
        BstNode* node = root_;
        while (node) { parent = node; node = (key < node->key) ?
node->left : node->right; }
        BstNode* fresh = new BstNode(key);
        fresh->parent = parent;
        if (!parent) root_ = fresh;
        else if (key < parent->key) parent->left = fresh;
        else parent->right = fresh;
        return fresh;
    }
    BstNode* root() const { return root_; }
    vector<int> inorder() const { vector<int> result;
inorderWalk(root_, result); return result; }
private:
    BstNode* root_ = nullptr;
    static void destroy(BstNode* node) {
        if (!node) return;
        destroy(node->left); destroy(node->right); delete node;
    }
};

// ----- red-black tree (A6-A10) -----
enum class Color { Red, Black };

struct RbNode {
    int key = 0;
    Color color = Color::Black;
    RbNode* left = nullptr;
    RbNode* right = nullptr;
    RbNode* parent = nullptr;
    int size = 1;           // A11/A12 augmentation:
nodes in this subtree
};

class RbTree {
public:

```

```

RbTree() {
    nil_ = new RbNode();
    nil_>color = Color::Black;
    nil_>size = 0; // the sentinel contributes
nothing to any count
    nil_>left = nil_>right = nil_>parent = nil_;
    root_ = nil_;
}
~RbTree() { destroy(root_); delete nil_; }
RbTree(const RbTree&) = delete;
RbTree& operator=(const RbTree&) = delete;

void leftRotate(RbNode* pivot);
// A6
void rightRotate(RbNode* pivot);
// A6
RbNode* insert(int key); // ordinary BST
insert + fixup
void insertFixup(RbNode* node);
// A7
void rbTransplant(RbNode* oldSubtree, RbNode* newSubtree);
// A8
void erase(RbNode* target);
// A9
void deleteFixup(RbNode* node);
// A10
RbNode* osSelect(RbNode* node, int rank) const;
// A11
int osRank(RbNode* node) const;
// A12

RbNode* minimum(RbNode* node) const { while (node->left !=
nil_) node = node->left; return node; }
RbNode* find(int key) const {
    RbNode* node = root_;
    while (node != nil_ && node->key != key) node = (key <
node->key) ? node->left : node->right;
    return node == nil_ ? nullptr : node;
}
RbNode* root() const { return root_; }
RbNode* nil() const { return nil_; }
int size() const { return root_>size; }

```

```

void inorder(RbNode* node, vector<int>& out) const {
    if (node == nil_) return;
    inorder(node->left, out); out.push_back(node->key);
    inorder(node->right, out);
}

vector<int> inorder() const { vector<int> result;
inorder(root_, result); return result; }

private:
    RbNode* root_ = nullptr;
    RbNode* nil_ = nullptr;
    void destroy(RbNode* node) {
        if (node == nil_) return;
        destroy(node->left); destroy(node->right); delete node;
    }

    // Recompute one node's augmented data from its children.
    Called after every
    // structural change -- this is step 4 of CLRS's four-step
    augmentation method.
    void pull(RbNode* node) { if (node != nil_) node->size = node-
>left->size + node->right->size + 1; }
    // Recompute from `node` up to the root. After a structural
    change, EVERY ancestor
    // of the deepest changed node has a stale count, so one
    bottom-up sweep
    // fixes them all -- and it is O(lg n), the same as the
    operation itself.
    void pullUp(RbNode* node) { while (node != nil_) { pull(node);
node = node->parent; } }
};

```

## A1 INORDER-TREE-WALK

*Pseudocode: §1, "Inorder traversal".*

```

// Appends this subtree's keys to `out` in SORTED order.
//
// `out` is a reference parameter (Weiss 1.5.3, p.26): the
recursion must all
// append to ONE vector. Returning a vector by value from each call
instead would
// be  $\Theta(n \lg n)$  work in copies for a balanced tree, and
 $\Theta(n^2)$  for a
// degenerate one.
void Bst::inorderWalk(BstNode* node, vector<int>& out) const {
    if (node != nullptr) {                                // 1  if x != NIL
        inorderWalk(node->left, out);                      // 2      INORDER-
TREE-WALK(x.left)
        out.push_back(node->key);                          // 3      print
x.key
        inorderWalk(node->right, out);                    // 4      INORDER-
TREE-WALK(x.right)
    }
}

```

**Complexity.**  $\Theta(n)$  — every node is visited exactly once, and the two recursive calls cost  $\Theta(1)$  of bookkeeping each. Space is  $\Theta(h)$  for the call stack:  $\Theta(\lg n)$  balanced,  $\Theta(n)$  degenerate (toolkit §6).

**The invariant that makes it work** is the BST property itself: everything in `x.left` is  $\leq$  `x.key`, everything in `x.right` is  $\geq$  `x.key`. Visiting left-node-right therefore emits keys in nondecreasing order. Swap two lines and you get preorder or postorder — and neither is sorted.

## A2 TREE-SEARCH, ITERATIVE-TREE-SEARCH, TREE-MINIMUM, TREE-MAXIMUM

*Pseudocode: §1, "Querying".*

```

// Recursive form -- a direct transcription.
BstNode* Bst::search(BstNode* node, int key) const {
    if (node == nullptr || key == node->key) return node;    //
1-2
    if (key < node->key) return search(node->left, key);      // 3-
4
    else return search(node->right, key);                    // 5

```

```

}

// Iterative form. Both recursive calls above are TAIL calls, so
// this conversion
// is mechanical -- and it matters here more than usual, because h
// can be n on a
// degenerate tree, and Theta(n) recursion depth is a segfault
// (toolkit 6).
BstNode* Bst::iterativeSearch(BstNode* node, int key) const {
    while (node != nullptr && key != node->key) {           // 1
        if (key < node->key) node = node->left;              //
2-3
        else node = node->right;                            // 4
    }
    return node;                                           // 5
    nullptr if absent
}

// TREE-MINIMUM: "the leftmost node". No comparison with k at all -
// the
// structure alone answers the question.
BstNode* Bst::minimum(BstNode* node) const {
    while (node != nullptr && node->left != nullptr) node = node-
>left;
    return node;
}
BstNode* Bst::maximum(BstNode* node) const {
    while (node != nullptr && node->right != nullptr) node = node-
>right;
    return node;
}

```

**Complexity.**  $O(h)$  for all four. That is  $O(\lg n)$  on a balanced tree and  $\Theta(n)$  on a degenerate one — which is precisely why the rest of this module exists. A BST's  $O(\lg n)$  is a property of the tree's shape, not of the algorithm, and nothing in TREE-SEARCH maintains that shape.

**Randomly built BSTs are fine on average:** expected height is  $\approx 4.311 \ln n$ , so random insertion order gives  $O(\lg n)$  expected. **Sorted insertion order gives a linked list.** Real data is very often sorted.



## A3 TREE-SUCCESSOR

Pseudocode: §1, "Querying".

```
// The next key in sorted order, or nullptr if x is the maximum.
// Two genuinely different cases, and the second is the one people
forget.
BstNode* Bst::successor(BstNode* node) const {
    // CASE A: x has a right subtree. The successor is the SMALLEST
thing bigger
    // than x, which is the leftmost node of that subtree.
    if (node->right != nullptr) return minimum(node->right);    //
1-2

    // CASE B: no right subtree. Everything below x is smaller than
x, so the
    // successor must be an ANCESTOR -- specifically the lowest
ancestor whose
    // LEFT subtree contains x. Climb while x is a right child.
    BstNode* ancestor = node->parent;
    // 4
    while (ancestor != nullptr && node == ancestor->right) {
        // 5
        node = ancestor;
    // 6
        ancestor = ancestor->parent;
        // 7
    }
    return ancestor;                                            //
8 nullptr if x was the maximum
}
```

**Complexity.**  $O(h)$  — one descent or one ascent, never both.

**Why this is worth knowing beyond trees:** repeatedly calling `TREE-SUCCESSOR` from the minimum walks the whole tree in sorted order in  $\Theta(n)$  total (each edge is traversed at most twice), using  $O(1)$  extra space rather than the  $O(h)$  stack of `INORDER-TREE-WALK`. That is exactly what `std::map::iterator::operator++` does, and it is why iterating a `map` needs no stack.

## A4 TRANSPLANT

Pseudocode: §1, "Deletion".

```
// Replace the subtree rooted at u with the subtree rooted at v.
// TRANSPLANT does NOT touch v's children -- it only fixes the link
// from above,
// plus v's parent pointer. Everything else is the caller's job.
void Bst::transplant(BstNode* oldSubtree, BstNode* newSubtree) {
    if (oldSubtree->parent == nullptr)          root_ = newSubtree;
        // 1 u was the root
    else if (oldSubtree == oldSubtree->parent->left)
oldSubtree->parent->left = newSubtree; // 2 u was a left child
    else                                     oldSubtree->parent->right =
newSubtree; // 3 u was a right child
        if (newSubtree != nullptr) newSubtree->parent = oldSubtree-
>parent; // 4 NULL-CHECKED here...
        // ...but the red-black version (A8) drops the check, because
        // there v can be
        // the SENTINEL and RB-DELETE-FIXUP genuinely needs nil's
        // parent set.
        // That one difference is the reason two nearly identical
        // procedures exist.
}
```

**Complexity.**  $\Theta(1)$ . Three pointer writes at most.

**This is the abstraction that makes `TREE-DELETE` readable.** Without it, delete is a thicket of "was z the root / a left child / a right child" tests repeated in every case.

## A5 TREE-DELETE

Pseudocode: §1, "Deletion — three shapes, four code paths".

```
void Bst::erase(BstNode* target) {
    if (target->left == nullptr) { // 1 CASE 1:
no left child
        transplant(target, target->right); // 2
(covers "no children" too)
    } else if (target->right == nullptr) { // 3 CASE 2:
exactly one child, on the left
        transplant(target, target->left); // 4
    } else {
```

```

        // Two children. Replace z with its SUCCESSOR y, which is
the minimum of
        // z's right subtree and therefore has NO LEFT CHILD --
that is what makes
        // it removable by a single transplant.
        BstNode* successorNode = minimum(target->right);
    // 5
        if (successorNode != target->right) {
    // 6 CASE 4: y is deeper than z.right
            transplant(successorNode, successorNode->right);
        // 7 lift y out of where it lives
            successorNode->right = target->right;
    // 8 give y all of z's right subtree
            successorNode->right->parent = successorNode;
        // 9
        }
        transplant(target, successorNode);
    // 10 CASE 3 joins here: y takes z's slot
        successorNode->left = target->left;
    // 11
        successorNode->left->parent = successorNode;
        // 12
    }
    delete target; // the pseudocode ends at line 12; C++ makes
you free the node
}

```

**Complexity.**  $O(h)$  — dominated by `TREE-MINIMUM`.

**The 4th-edition change worth knowing.** Older presentations *copy y's key into z* and then delete `y`. CLRS 4e **moves the node** instead. The difference is invisible on paper and load-bearing in practice: if a client holds a pointer (or an iterator) to a node, copying keys silently changes what that pointer refers to, whereas moving nodes keeps every surviving pointer valid. This is exactly the guarantee `std::map` gives — `erase` invalidates only the erased element's iterator — and it is why the standard could later expose `extract()`.

**Deletion is not commutative.** Deleting `x` then `y` can produce a different tree shape than `y` then `x`. Nothing is wrong; the BST property just does not pin down a unique shape.

## A6 LEFT-ROTATE and RIGHT-ROTATE

Pseudocode: §2, "Rotations".

```
// Rotate `pivot` down-left and its right child up. PRESERVES the
// BST property:
//
//      pivot          newParent
//      / \           / \
//      a  newParent -> pivot  c
//      / \           / \
//      b  c          a  b
//
// Before: a < pivot < b < newParent < c.   After: unchanged.   New
// shape, same order.
void RbTree::leftRotate(RbNode* pivot) {
    RbNode* newParent = pivot->right;          // 1  assumes pivot-
    >right != nil
    pivot->right = newParent->left;              // 2  its left
    subtree becomes pivot's right
    if (newParent->left != nil_) newParent->left->parent = pivot;
    // 3
    newParent->parent = pivot->parent;           // 4  link newParent
    to pivot's parent
    if (pivot->parent == nil_)                   root_ = newParent;
    // 5
    else if (pivot == pivot->parent->left)       pivot->parent->left
    = newParent;    // 6
    else                                         pivot->parent->right
    = newParent;    // 7
    newParent->left = pivot;                     // 8  put pivot on
    newParent's left
    pivot->parent = newParent;                   // 9
    // AUGMENTATION MAINTENANCE (Theorem 17.1): both nodes changed
    children, so
    // their subtree sizes must be recomputed -- `pivot` FIRST,
    because it is now
    // newParent's child and newParent's size depends on it. Order
    matters.
    pull(pivot);
    pull(newParent);
}
```

```

// The exact mirror image. Writing it out rather than
// parameterising by direction
// is deliberate: the mirrored version is where transcription bugs
// hide, and
// having both side by side makes them visible.
void RbTree::rightRotate(RbNode* pivot) {
    RbNode* newParent = pivot->left;
    pivot->left = newParent->right;
    if (newParent->right != nil_) newParent->right->parent = pivot;
    newParent->parent = pivot->parent;
    if (pivot->parent == nil_)          root_ = newParent;
    else if (pivot == pivot->parent->right) pivot->parent->right
= newParent;
    else                               pivot->parent->left
= newParent;
    newParent->right = pivot;
    pivot->parent = newParent;
    pull(pivot);
    pull(newParent);
}

```

**Complexity.**  $\Theta(1)$  — a fixed number of pointer updates, regardless of subtree size. That is the entire reason rebalancing can be cheap: **a rotation restructures an arbitrarily large tree in constant time.**

## A7 RB-INSERT-FIXUP

*Pseudocode: §2, "Insertion".*

```

// A new node is inserted RED (so black-heights are untouched) and
// may therefore
// violate property 4: "a red node has two black children".
//
// LOOP INVARIANT, three parts:
//   a. z is red.
//   b. if z.p is the root, then z.p is black.
//   c. at most one red-black property is violated, and if so it is
//      either
//         property 2 (root is black, when z IS the root) or property
//         4 (z and z.p
//         are both red).

```

```

void RbTree::insertFixup(RbNode* node) {
    while (node->parent->color == Color::Red) {
        // 1
        if (node->parent == node->parent->parent->left) {
            // 2 parent is a LEFT child
            RbNode* uncle = node->parent->parent->right;
            // 3 y = z's UNCLE
            if (uncle->color == Color::Red) {
                // 4 ---- CASE 1: red uncle
                node->parent->color = Color::Black;
                // 5
                uncle->color = Color::Black;
                // 6
                node->parent->parent->color = Color::Red;
                // 7 push blackness DOWN
                node = node->parent->parent;
                // 8 violation moves up TWO levels
                // No rotation at all. This is the only case that
                iterates, and
                // it climbs by 2, which is why the loop runs O(lg
                n) times.
            } else {
                if (node == node->parent->right) {
                    // 10 ---- CASE 2: black uncle, zig-zag
                    node = node->parent;
                    // 11
                    leftRotate(node);
                    // 12 straighten into Case 3
                }
                node->parent->color = Color::Black;
                // 13 ---- CASE 3: black uncle, straight
                node->parent->parent->color = Color::Red;
                // 14
                rightRotate(node->parent->parent);
                // 15 loop EXITS after this: z->p
                //
                is now black, so the test fails
            }
        } else {
            // 16
            mirror image
            RbNode* uncle = node->parent->parent->left;
            if (uncle->color == Color::Red) {

```

```

        node->parent->color = Color::Black;
        uncle->color = Color::Black;
        node->parent->parent->color = Color::Red;
        node = node->parent->parent;
    } else {
        if (node == node->parent->left) { node = node-
>parent; rightRotate(node); }
        node->parent->color = Color::Black;
        node->parent->parent->color = Color::Red;
        leftRotate(node->parent->parent);
    }
}
}
root_->color = Color::Black; // 17
restores property 2 unconditionally
}

// The insert itself: an ordinary BST insert, coloured red, then
fix up.
RbNode* RbTree::insert(int k) {
    RbNode* node = new RbNode();
    node->key = k;
    node->left = node->right = node->parent = nil_;
    node->color = Color::Red;
    node->size = 1;

    RbNode* uncle = nil_;
    RbNode* x = root_;
    while (x != nil_) { uncle = x; ++x->size; x = (k < x->key) ? x-
>left : x->right; }
    // ^^^^^^^^^ the augmentation, updated
on the way DOWN:
    // every node on the search path gains
one descendant.
    node->parent = uncle;
    if (uncle == nil_) root_ = node;
    else if (k < uncle->key) uncle->left = node;
    else uncle->right = node;

    insertFixup(node);
    return node;
}

```

**Complexity.**  $O(\lg n)$ , with at most 2 rotations. The loop only *iterates* in Case 1, which moves  $z$  up two levels, so it runs at most  $\lfloor h/2 \rfloor$  times. Cases 2 and 3 each rotate and then terminate.

**Why the tree is  $O(\lg n)$  tall at all** is Lemma 13.1: a red-black tree with  $n$  internal nodes has height  $h \leq 2 \lg(n + 1)$ . The proof is two steps — a subtree rooted at  $x$  contains at least  $2^{\text{bh}(x)} - 1$  internal nodes (induction), and at least half the nodes on any root-to-leaf path are black (property 4) — so  $n \geq 2^{h/2} - 1$ .

## A8 RB-TRANSPLANT

*Pseudocode: §2, "Deletion".*

```
void RbTree::rbTransplant(RbNode* oldSubtree, RbNode* newSubtree) {
    if (oldSubtree->parent == nil_)        root_ = newSubtree;
        // 1
    else if (oldSubtree == oldSubtree->parent->left)
oldSubtree->parent->left = newSubtree;    // 2
    else                                oldSubtree->parent->right =
newSubtree;    // 3
    newSubtree->parent = oldSubtree->parent;
        // 4 UNCONDITIONAL -- no null check
    // This single line is the whole difference from A4's
    TRANSPLANT. v may be
    // the sentinel nil_, and RB-DELETE-FIXUP will climb from x ==
    nil_ using
    // exactly this parent pointer. Guarding it with `if (v !=
    nil_)` -- the
    // instinct every reader has -- breaks deletion in a way that
    only shows up
    // on specific shapes. It is the single most common red-black
    bug.
}
```

**Complexity.**  $\Theta(1)$ .

## A9 RB-DELETE

*Pseudocode: §2, "Deletion".*



```

void RbTree::erase(RbNode* target) {
    RbNode* removed = target;
1  y is the node actually REMOVED
    Color removedWasRed = removed->color;
    or MOVED within the tree
    RbNode* fixupAt = nil_;
takes y's place
    RbNode* from = nil_;
node whose subtree changed

    if (target->left == nil_) {
        fixupAt = target->right;
4
        from = target->parent;
        rbTransplant(target, target->right);
    } else if (target->right == nil_) {
        fixupAt = target->left;
6
        from = target->parent;
        rbTransplant(target, target->left);
    } else {
        removed = minimum(target->right);
7  y = z's successor
        removedWasRed = removed->color;
        fixupAt = removed->right;
9
        if (removed != target->right) {
10 y is farther down
            from = removed->parent;
y's old parent lost a subtree
            rbTransplant(removed, removed->right);
11
            removed->right = target->right;
12
            removed->right->parent = removed;
        } else {
            fixupAt->parent = removed;
// 13 "in case x is nil_" -- and it
            from = removed;
matters, see A8
    }
}

```

```

        rbTransplant(target, removed);                                //
14
        removed->left = target->left;                                //
15
        removed->left->parent = removed;
        removed->color = target->color;                                //
16 y INHERITS z's colour, so no
    }                                                                //
violation is introduced HERE

    // AUGMENTATION MAINTENANCE. Do NOT try to decrement counts on
the way down:
    // in the two-children case the successor MOVES, so the set of
nodes whose
    // subtree changed is not simply "the ancestors of z", and an
ad-hoc
    // decrement double-counts. One bottom-up recompute from the
deepest changed
    // node is both simpler and correct -- and after the
transplants the parent
    // chain from `from` passes through y, so this covers y too.
pullUp(from);

    // Removing a RED node cannot break anything: black-heights are
unchanged and
    // no red-red pair can be created. Removing a BLACK node
removes one black
    // from every path through it -- that is the violation the
fixup repairs.
    // (deleteFixup's rotations call pull() themselves, so sizes
stay correct.)
    if (removedWasRed == Color::Black) deleteFixup(fixupAt);    //
17-18
    delete target;
}

```

**Complexity.**  $O(\lg n)$ , dominated by `TREE-MINIMUM` and the fixup.

## A10 RB-DELETE-FIXUP

*Pseudocode: §2, "Deletion".*

```

// x carries an "extra black" -- the black that was lost when y was
removed.
// Think of x as holding 2 units of blackness (doubly black) or, if
x is red,
// 1.5 (red-and-black). The loop pushes that extra unit up the tree
until it can
// be absorbed: by a red node (which just turns black), or by the
root (where it
// simply evaporates, since every path loses one black equally).
void RbTree::deleteFixup(RbNode* node) {
    while (node != root_ && node->color == Color::Black) {
        // 1
        if (node == node->parent->left) {
            // 2
            RbNode* sibling = node->parent->right;
            // 3 w = x's SIBLING
            // w can never be nil_: x is doubly black, so the path
through x
            // already has >= 2 blacks, so x's sibling subtree must
be non-empty.
            if (sibling->color == Color::Red) {
                // 4 -- CASE 1
                sibling->color = Color::Black;
                // 5
                node->parent->color = Color::Red;
                leftRotate(node->parent);
                // 7
                sibling = node->parent->right; // ...and now w is
BLACK: cases 2/3/4 follow
            }
            if (sibling->left->color == Color::Black &&
                sibling->right->color == Color::Black) {
                // 9 -- CASE 2
                sibling->color = Color::Red;
                // 10 take a black off w
                node = node->parent;
                // and push the extra

                // black up to the parent
                // THE ONLY case that iterates. It climbs one
level, so the loop
                // runs O(lg n) times -- and does no rotation.
            }
        }
    }
}

```

```

    } else {
        if (sibling->right->color == Color::Black) {
            // 13 -- CASE 3
            sibling->left->color = Color::Black;
            // 14
            sibling->color = Color::Red;
            rightRotate(sibling);
            // 16
            sibling = node->parent->right;    // turn Case 3
into Case 4
        }
        sibling->color = node->parent->color;
        // 18 -- CASE 4
        node->parent->color = Color::Black;
        // 19
        sibling->right->color = Color::Black;
        leftRotate(node->parent);
        // 21
        node = root_;
        // DONE: forces exit
    }
} else {
    // mirror image
    RbNode* sibling = node->parent->left;
    if (sibling->color == Color::Red) {
        sibling->color = Color::Black; node->parent->color
= Color::Red;
        rightRotate(node->parent); sibling = node->parent-
>left;
    }
    if (sibling->right->color == Color::Black && sibling-
>left->color == Color::Black) {
        sibling->color = Color::Red; node = node->parent;
    } else {
        if (sibling->left->color == Color::Black) {
            sibling->right->color = Color::Black; sibling-
>color = Color::Red;
            leftRotate(sibling); sibling = node->parent-
>left;
        }
        sibling->color = node->parent->color;
        node->parent->color = Color::Black;
    }
}

```

```

        sibling->left->color = Color::Black;
        rightRotate(node->parent);
        node = root_;
    }
}
}
node->color = Color::Black;    // absorb the extra black; also
fixes a red x
}

```

**Complexity.**  $O(\lg n)$  time, at most 3 rotations. Cases 1, 3 and 4 each rotate and then either terminate or lead directly to termination; only Case 2 loops, and it climbs one level per iteration.

**Compare with insertion: at most 2 rotations, 3 cases. Deletion: at most 3 rotations, 4 cases.** Both are  $O(\lg n)$  with a constant number of structural changes — and that bounded number of rotations is exactly what makes red-black trees suitable for augmentation (Theorem 17.1: augmenting is cheap precisely because only  $O(1)$  nodes change shape per operation).

## A11 OS-SELECT

*Pseudocode: §3, "Order-statistic trees".*

```

// The i-th smallest key in x's subtree, 1-indexed. Requires the
`size`
// augmentation, which leftRotate/rightRotate/insert/erase above
maintain.
RbNode* RbTree::osSelect(RbNode* node, int rank) const {
    if (node == nil_) return nullptr;
    int leftPlusSelf = node->left->size + 1;          // 1  x's
own rank WITHIN ITS SUBTREE
    //      ^^^^^^^^^^^^^^^ this is why nil_->size == 0: no special
case needed
    if (rank == leftPlusSelf)      return node;          // 2
    else if (rank < leftPlusSelf)  return osSelect(node->left,
rank);          // 3
    else                          return osSelect(node->right, rank -
leftPlusSelf); // 4
    //                                  ^^^^^ RE-BASE: the
r elements at
    //                                  or before x are gone from consideration.
Forgetting this
    //                                  subtraction is the same bug as in
quickselect (M05 A10).
}

```

**Complexity.**  $O(\lg n)$  — one root-to-node descent,  $\Theta(1)$  per level.

## A12 OS-RANK

*Pseudocode:* §3, "Order-statistic trees".

```

// The position of x in the tree's sorted order, 1-indexed. The
// inverse of osSelect.
int RbTree::osRank(RbNode* node) const {
    int rank = node->left->size + 1;           // 1  rank within
    x's own subtree
    const RbNode* climber = node;             // 2
    while (climber != root_) {                 // 3  climb to
    the root
        if (climber == climber->parent->right) // 4
        y is a RIGHT child, so its parent
            rank += climber->parent->left->size + 1; // 5  and
        the parent's entire LEFT subtree
                                                    // 6  all precede x --
        add them in
        climber = climber->parent;              // 6
    }
    return rank;                               // 7
}

```

**Complexity.**  $O(\lg n)$  — one node-to-root ascent.

**These two functions are the payoff of Theorem 17.1**, CLRS's four-step augmentation method: (1) choose the underlying structure — red-black tree; (2) choose the extra data — `size`; (3) verify it can be maintained — `size(x) = size(left) + size(right) + 1` depends only on the children, so a rotation fixes it in  $\Theta(1)$ ; (4) develop the new operations. **Step 3 is the whole theorem: if the augmented field at a node is computable from its children's fields, it costs nothing asymptotically to maintain.**

## A13 INTERVAL-SEARCH

*Pseudocode: §3, "Interval trees".*

```

struct Interval { long long low, high; };

// Interval trichotomy (CLRS): exactly one of these holds for any i
// and j --
// (a) they overlap, (b) i.high < j.low, (c) j.high < i.low.
// So "overlap" is the NEGATION of the two easy cases, which is why
// the test is
// written this way rather than as four comparisons.
static bool overlaps(const Interval& lhs, const Interval& rhs) {

```

```

    return lhs.low <= rhs.high && rhs.low <= lhs.high;
}

struct IntervalNode {
    Interval interval{0, 0};
    long long maxHigh = LLONG_MIN;    // the augmentation: max
    `high` in this subtree
    Color color = Color::Black;
    IntervalNode *left = nullptr, *right = nullptr, *parent =
    nullptr;
};

// Keyed by interval.low; augmented with maxHigh = max over the
// subtree.
IntervalNode* intervalSearch(IntervalNode* root, IntervalNode* nil,
    const Interval& query) {
    IntervalNode* node = root;
    // 1
    while (node != nil && !overlaps(query, node->interval)) {
        // 2
        if (node->left != nil && node->left->maxHigh >= query.low)
            // 3
            node = node->left;
        // 4
        else
            node = node->right;
        // 5
    }
    return node == nil ? nullptr : node;
    // 6
}

```

**Complexity.**  $O(\lg n)$  — a single root-to-leaf descent with no backtracking.

**Why going left is safe (Theorem 17.2), and it is the only subtle part.** The loop makes one choice per level and never backtracks, so the choice must be *provably* right:

- **If it goes left** ( $x.\text{left}.\text{max} \geq i.\text{low}$ ): either the left subtree contains an overlapping interval, or it does not — and in the latter case, CLRS proves the *right* subtree cannot contain one either. So going left loses nothing.
- **If it goes right** ( $x.\text{left}.\text{max} < i.\text{low}$ ): every interval in the left subtree ends



before  $i$  begins, so none of them can overlap. Going right loses nothing.

**One more property worth remembering:** it returns *some* overlapping interval, not all of them and not a specific one. Finding all  $k$  overlaps takes  $O(k \lg n)$  by repeated search-and-delete, or  $O(\lg n + k)$  with a modified traversal.

## A14 B-TREE-SEARCH

*Pseudocode: §4, "Searching".*

```
// Minimum degree t: every node except the root holds between t-1
// and 2t-1 keys,
// and between t and 2t children. `t` is a template parameter so
// the arrays can
// be fixed-size -- a real B-tree sizes t so that one node fills
// one disk block.
template <int MinDegree>                                // CLRS calls this `t`
struct BTreeNode {
    int keyCount = 0;                                     // number of keys
    // currently stored (CLRS: x.n)
    bool leaf = true;
    array<int, 2 * MinDegree - 1> keys{};                // 1-indexed in the
    // pseudocode;
    array<BTreeNode*, 2 * MinDegree> children{}; // 0-indexed here,
    // hence the -1s below
};

// Returns (node, index) or (nullptr, -1). `pair` rather than two
// out-parameters:
// the two values are meaningless apart.
template <int MinDegree>
pair<BTreeNode<MinDegree>*, int> bTreeSearch(BTreeNode<MinDegree>*
node, int key) {
    int i = 0;                                           // 1
    // (pseudocode: i = 1)
    while (i < node->keyCount && key > node->keys[i]) ++i; // 2-3
    // linear scan WITHIN the node
    // A binary search here would be  $O(\lg t)$  instead of  $O(t)$ , but  $t$ 
    // is chosen so
    // a node fits one disk block: the scan is in RAM and free next
    // to the I/O.
```

```

        if (i < node->keyCount && key == node->keys[i]) return {node,
i};    // 4-5 found
        if (node->leaf) return {nullptr, -1};           // 6-7
absent
        // 8 DISK-READ(x.c_i) -- in a real B-tree this is the
expensive line, and
        // it is the only one the complexity analysis counts.
        return bTreeSearch(node->children[i], key);    // 9
    }

```

**Complexity.**  $O(\log_t n)$  disk accesses,  $O(t \cdot \log_t n)$  CPU.

*Measured: 200 000 keys inserted into a B-tree with  $t = 50$  gives **height 2**; the same keys with  $t = 3$  give **height 9**. Same data, same algorithm, one parameter — and that parameter is the base of the logarithm.*

**The point of a B-tree is the base of that logarithm.** Theorem 18.1:  $n \leq \log_t((n+1)/2)$ . With  $t = 1001$ , a tree holding **one billion** keys has height **2** — a root plus two levels — so any key is three disk reads away. A red-black tree over the same data is 30 levels deep, and on disk that is 30 seeks. **The structure exists because a disk seek costs  $\sim 10^5$  times a memory access**, so the design minimises seeks even at the cost of far more CPU work per node.

## A15 B-TREE-SPLIT-CHILD

*Pseudocode: §4, "Insertion".*

```

// x is NONFULL, x->c[i] is FULL (2t-1 keys). Split that child into
two nodes of
// t-1 keys each and push its MEDIAN key up into x. x gains one key
and one child.
template <int MinDegree>
void bTreeSplitChild(BTreeNode<MinDegree>* parent, int childIndex)
{
    constexpr int t = MinDegree;           //
    local shorthand, CLRS's `t`
    BTreeNode<t>* fullChild = parent->children[childIndex];
    // 1
    BTreeNode<t>* newSibling = new BTreeNode<t>();           //
    2 ALLOCATE-NODE()
    newSibling->leaf = fullChild->leaf;           // 3
}

```

```

    newSibling->keyCount = t - 1; // 4

    for (int j = 0; j < t - 1; ++j) // 5 newSibling takes
the GREATEST t-1 keys
        newSibling->keys[j] = fullChild->keys[j + t]; // 6
    if (!fullChild->leaf) // 7
        for (int j = 0; j < t; ++j) // 8
...and the matching t children
        newSibling->children[j] = fullChild->children[j + t];
// 9

    const int median = fullChild->keys[t - 1]; //
saved before it is overwritten
    fullChild->keyCount = t - 1; // 10 fullChild keeps
the SMALLEST t-1 keys
// and the median
moves up

    for (int j = parent->keyCount; j >= childIndex + 1; --j) //
11 shift children right
        parent->children[j + 1] = parent->children[j]; //
12 -- DOWNWARD loop, because
    parent->children[childIndex + 1] = newSibling; //
13 the ranges overlap and an
    for (int j = parent->keyCount - 1; j >= childIndex; --j) //
14 upward loop would overwrite
        parent->keys[j + 1] = parent->keys[j]; //
15 entries before reading them
    parent->keys[childIndex] = median; // 16 the
MEDIAN moves up into parent
    parent->keyCount = parent->keyCount + 1; // 17
    // 18 DISK-WRITE(y); DISK-WRITE(z); DISK-WRITE(x)
}

```

**Complexity.**  $\Theta(t)$  CPU,  $\mathcal{O}(1)$  disk accesses — three writes, no reads beyond the two nodes already in memory.

## A16 B-TREE-INSERT and B-TREE-SPLIT-ROOT

*Pseudocode: §4, "Insertion — split full nodes on the way down".*

```

// Forward declarations: BTree::insert calls these before they are
defined
// (A15 and A17 give the definitions). A template must be DECLARED
before use.

template <int MinDegree> void bTreeSplitChild(BTreeNode<MinDegree>*
node, int childIndex);
template <int MinDegree> void
bTreeInsertNonfull(BTreeNode<MinDegree>* node, int key);

template <int MinDegree>
class BTree {
public:
    BTree() : root_(new BTreeNode<MinDegree>()) {}
    ~BTree() { destroy(root_); }
    BTree(const BTree&) = delete;
    BTree& operator=(const BTree&) = delete;

    void insert(int key);
    bool contains(int key) const { return bTreeSearch(root_,
key).first != nullptr; }
    int height() const { int h = 0; for (auto* node = root_; !node-
>leaf; node = node->children[0]) ++h; return h; }
    BTreeNode<MinDegree>* root() const { return root_; }
private:
    BTreeNode<MinDegree>* root_;
    BTreeNode<MinDegree>* splitRoot();
    static void destroy(BTreeNode<MinDegree>* node) {
        if (!node) return;
        if (!node->leaf) for (int i = 0; i <= node->keyCount; ++i)
destroy(node->children[i]);
        delete node;
    }
};

// B-TREE-SPLIT-ROOT: the ONLY operation that makes a B-tree
taller.

template <int MinDegree>
BTreeNode<MinDegree>* BTree<MinDegree>::splitRoot() {
    BTreeNode<MinDegree>* newRoot = new BTreeNode<MinDegree>(); //
1
    newRoot->leaf = false; //
2

```

```

    newRoot->keyCount = 0; //
3
    newRoot->children[0] = root_; // 4 the old root becomes
newRoot's only child
    root_ = newRoot; //
5
    bTreeSplitChild<MinDegree>(newRoot, 0);
// 6 then split it, giving newRoot one key and
two children
    return newRoot; //
7
}

template <int MinDegree>
void BTree<MinDegree>::insert(int key) {
    BTreeNode<MinDegree>* oldRoot = root_; // 1
    if (oldRoot->keyCount == 2 * MinDegree - 1) { // 2
root is full
        BTreeNode<MinDegree>* newRoot = splitRoot(); // 3
        bTreeInsertNonfull<MinDegree>(newRoot, key); // 4
    } else {
        bTreeInsertNonfull<MinDegree>(oldRoot, key); // 5
    }
}

```

**Complexity.**  $O(\log_t n)$  disk accesses,  $O(t \log_t n)$  CPU — one pass down, no pass back up.

**A B-tree grows at the TOP, not the bottom.** That single fact is why **all leaves stay at the same depth**: the tree gains a level only when the root splits, and at that instant *every* leaf gets one level deeper simultaneously. A BST grows at the bottom, which is exactly why its leaves end up at wildly different depths.

## A17 B-TREE-INSERT-NONFULL

*Pseudocode: §4, "Insertion".*

```

// PRECONDITION: x is not full. That precondition is maintained by
splitting any
// full child BEFORE descending into it -- the "proactive
splitting" that lets

```

```

// insertion finish in ONE downward pass, with no recursion back up
the tree.
template <int MinDegree>
void bTreeInsertNonfull(BTreeNode<MinDegree>* node, int key) {
    int i = node->keyCount - 1; // 1
    (pseudocode: i = x.n)
    if (node->leaf) { // 2
        while (i >= 0 && key < node->keys[i]) { // 3
            shift to make room
            node->keys[i + 1] = node->keys[i]; // 4
            --i;
        }
        node->keys[i + 1] = key; // 6 --
        this is INSERTION-SORT's
        node->keyCount = node->keyCount + 1; //
        inner loop (M01 A5), on one node
        // 8 DISK-WRITE(x)
    } else {
        while (i >= 0 && key < node->keys[i]) --i; // 9-10
        find the child to descend into
        ++i; // 11
        // 12 DISK-READ(x.c_i)
        if (node->children[i]->keyCount == 2 * MinDegree - 1) {
            // 13 child is FULL: split it NOW,
            bTreeSplitChild<MinDegree>(node, i); // 14
            while we are still here and node
            // is
            guaranteed to have room
            if (key > node->keys[i]) ++i; // 15 the
            split pushed a median up;
            //
            decide which half key belongs in
            bTreeInsertNonfull<MinDegree>(node->children[i], key); //
            17 tail call -> a loop, so
            //
            only O(1) nodes need be resident
        }
    }
}

```

**Complexity.**  $O(\log_t n)$  disk accesses,  $O(t \log_t n)$  CPU.

**Why split *before* descending rather than after overflowing.** The reactive approach — descend, insert, and split on the way back up if the node overflowed — needs a second pass upward and forces you to keep every node on the path in memory. Proactive splitting guarantees the parent always has room for a promoted median, so insertion is one downward pass with  $O(1)$  nodes resident. The tail-recursive call above makes that explicit: convert it to a `while` loop and the memory requirement is manifestly constant.

**The cost is splitting nodes that did not strictly need it** — a node that is full gets split even if the insertion would not have overflowed it. That is a deliberate trade of a little wasted work for a much simpler, single-pass, low-memory algorithm.

---

Next: [M09 — Amortized Analysis](#) (CLRS 16) — aggregate, accounting and potential methods; dynamic tables; and the tool that lets us say a sequence of  $n$  operations costs  $O(n)$  even when one of them costs  $\Theta(n)$ .